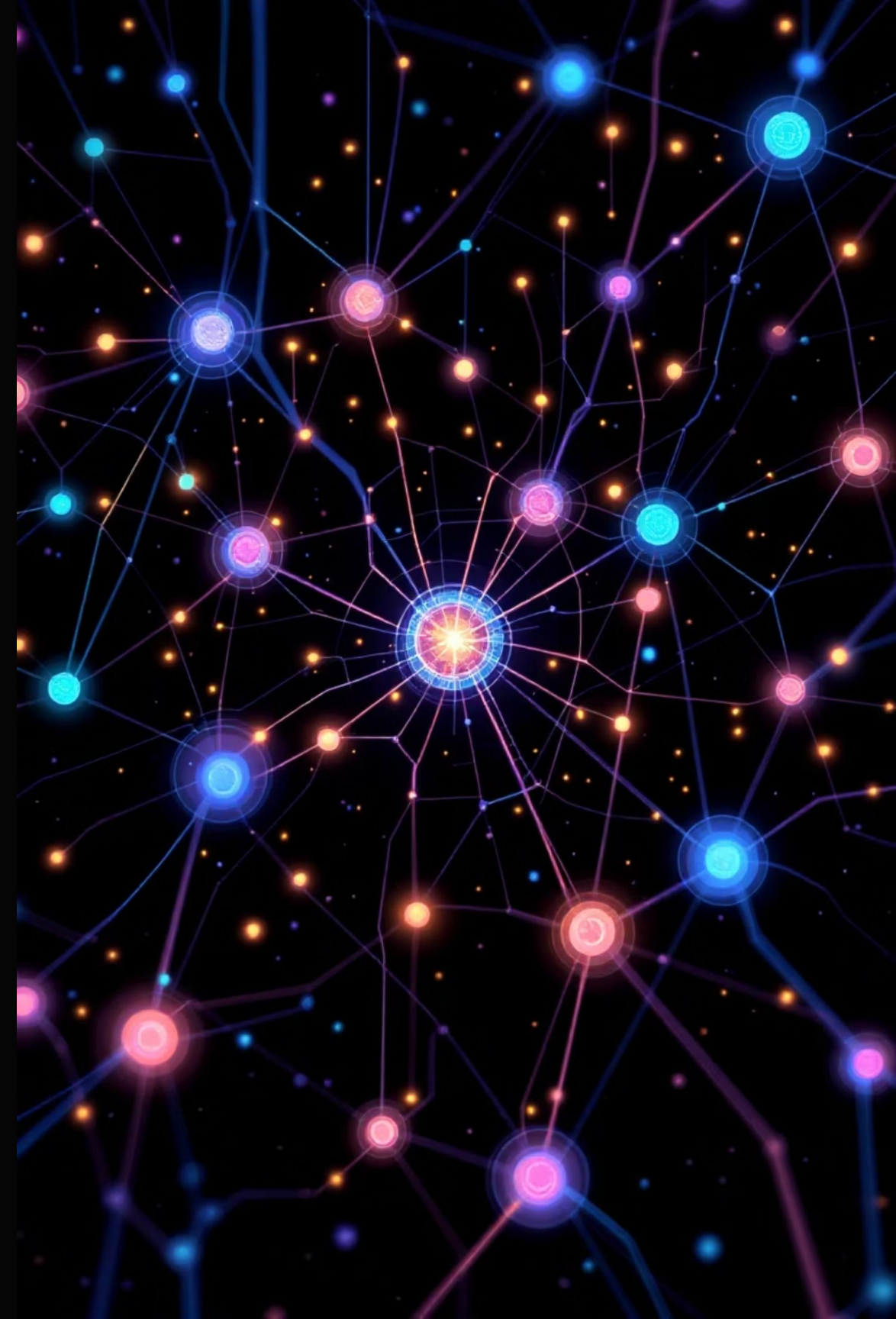


Understanding Word Representations & Convolutional Neural Networks

Delve into the fascinating world of natural language processing, exploring how computers comprehend and process human language through sophisticated word representations and powerful neural network architectures.



Navigating the Landscape of NLP: Our Exploration Agenda

This presentation will guide you through the fundamental concepts underpinning modern Natural Language Processing (NLP), from the basic building blocks of word representation to the advanced architectures of Convolutional Neural Networks.

01

From One-Hot to Distributed Representations

Understanding the evolution of how words are numerically encoded for machine comprehension.

02

Learning Word Representations with SVD

Exploring an early, foundational method for uncovering semantic relationships in language data.

03

Word2Vec: CBOW and Skip-Gram Models

Unpacking the revolutionary techniques that transformed word embeddings and their practical applications.

04

Introduction to Convolutional Neural Networks (CNNs)

Discovering how these powerful networks identify patterns, initially in images, and then in text.

05

Combining Word2Vec & CNNs

Seeing how these two innovations merge to achieve superior performance in text classification.

From One-Hot to Distributed Word Representations

Initially, words were represented using **one-hot vectors**, where each unique word in a vocabulary was assigned a unique, high-dimensional vector. For instance, in a vocabulary of 50,000 words, 'cat' might be $[0, 0, \dots, 1, \dots, 0]$, with a 1 at its specific index and 0 everywhere else. This method is simple but highly **sparse** and fundamentally lacks any notion of semantic similarity; 'cat' and 'dog' vectors are orthogonal, implying no relationship, which is untrue in human language.

The key **limitation** of one-hot encoding is its inability to capture word relationships or context. To overcome this, the field shifted towards **distributed representations**. These models embed words as dense, low-dimensional vectors (typically 50-300 dimensions) where the numerical proximity of vectors directly reflects the **semantic meaning and relationships** between words. This allows algorithms to understand that 'king' is similar to 'queen' and 'man' is similar to 'woman'.

One-hot encoding: Simple but semantically blind.

Distributed representations: Capturing meaning and context.

Learning Word Representations with SVD

Before the advent of neural embedding models, **Singular Value Decomposition (SVD)** played a significant role in learning word representations. SVD was typically applied to **word co-occurrence matrices**, which record how often words appear together within a certain window of text. For example, a row might represent a target word, and columns its neighbouring words, with cell values indicating their co-occurrence frequency.

By performing SVD on these large, sparse matrices, the dimensionality could be significantly **reduced**, transforming the high-dimensional word vectors into lower-dimensional, dense vectors. This process effectively **captures latent semantic structures** and relationships between words, as words that frequently co-occur will tend to have similar reduced-dimension representations. SVD was an **early but fundamental method**, laying a theoretical groundwork for later, more advanced neural network-based embedding techniques like Word2Vec.

Matrix Factorisation

SVD decomposes large co-occurrence matrices into components that reveal underlying patterns.

Dimensionality Reduction

Compresses high-dimensional data into more manageable, information-rich vectors.

Latent Semantic Analysis

Uncovers hidden semantic relationships between words based on their contextual usage.

Word2Vec: Revolutionising Word Embeddings

Word2Vec, introduced by a team led by Tomas Mikolov at Google in 2013, marked a pivotal moment in NLP. It's an efficient, predictive model for learning high-quality **word embeddings** from large text corpora. Unlike count-based methods like SVD on co-occurrence matrices, Word2Vec employs shallow neural networks to learn word associations from local context windows.

The core idea behind Word2Vec is to learn vector representations for words such that words appearing in similar contexts have similar vectors. It does this by either **predicting a target word from its surrounding context words** (Continuous Bag of Words, CBOW) or by **predicting the surrounding context words given a target word** (Skip-gram). This approach enables Word2Vec to capture nuanced **semantic and syntactic relationships**, famously demonstrated by vector arithmetic such as "king" - "man" + "woman" $\approx$ "queen".

Neural Network Approach

Uses shallow neural networks for efficient learning.



Contextual Prediction

Learns word meanings by predicting words from their neighbours.



Semantic Analogy

Captures surprising linguistic relationships through vector operations.

Continuous Bag of Words (CBOW) Model

The Continuous Bag of Words (CBOW) model within Word2Vec operates on the principle that the meaning of a word can be inferred from its surrounding context. Specifically, CBOW aims to **predict a target word based on the sum or average of the word vectors of its context words** within a defined window. The "bag-of-words" aspect implies that the order of the context words does not matter for the prediction, only their presence.

This model is particularly **efficient for frequent words** and typically leads to **faster training times** compared to the Skip-gram model, especially with larger datasets. For example, if the sentence is "The **cat** sat on the **mat**", and our context window is one word before and one word after, given context words like "cat" and "on", the CBOW model tries to predict the target word "mat". The model effectively averages the context vectors to create a combined context representation before making its prediction.

Example:

Given context words ["the", "cat", "on", "the"], the CBOW model attempts to predict the target word "**mat**". It aggregates the vectors for "the", "cat", "on", "the" to form a single context vector, then uses this to predict "mat".

Skip-Gram Model

In contrast to CBOW, the Skip-gram model approaches word embedding from the opposite direction. Instead of predicting a target word from its context, Skip-gram is designed to **predict the surrounding context words given a single target word**. For each word in the training corpus, the model selects a window of context words around it and then tries to predict each of these context words individually.

The Skip-gram model is generally considered **more effective for rare words** and can **capture diverse semantic contexts** better than CBOW. This is because it trains on more word-context pairs, allowing it to learn richer representations for less frequent words. For instance, if the target word is "mat", and the context window is two words before and two words after, the Skip-gram model would predict context words such as "the", "cat", "sat", and "on" from "mat" individually, creating multiple training examples per word.

Example:

Given the target word "**mat**", the Skip-gram model predicts surrounding context words like **["the", "cat", "on", "the"]**. Each context word forms a separate prediction task from the central "mat" word.

Word2Vec in Practice: Semantic Vector Space

The true power of Word2Vec lies in its ability to create a **semantic vector space** where words with similar meanings are located close to each other. This spatial arrangement allows for powerful operations:

- Words that are semantically related, such as "doctor" and "nurse," will have vectors that are numerically close.
- It enables **analogies** through vector arithmetic, like the famous "King - Man + Woman $\approx$ Queen" example, demonstrating that the model understands complex relationships beyond simple similarity.
- **Similarity measures** can be computed using metrics like cosine similarity between word vectors, quantifying how related two words are.

Word2Vec embeddings have become an indispensable component in numerous **Natural Language Processing (NLP) tasks**, significantly improving performance across the board. Their ability to represent words as rich, dense vectors has made them a standard input for various machine learning models.



Text Classification

Categorising documents or sentences based on content.



Document Clustering

Grouping similar documents together for analysis.



Named Entity Recognition

Identifying and classifying named entities (e.g., people, organisations, locations).



Recommendation Systems

Suggesting items based on textual descriptions and user preferences.

Introduction to Convolutional Neural Networks (CNNs)

Convolutional Neural Networks (CNNs) are a class of deep neural networks predominantly used for analysing visual imagery, but their utility extends far beyond. The defining characteristic of CNNs is their use of **convolutional layers**, which employ small matrices known as **kernel filters** (or simply kernels).

These kernels perform a mathematical operation called convolution, sliding over the input data (e.g., an image or a sequence of word embeddings) to detect **local patterns**. In image processing, kernels can identify features like edges, lines, or textures. In NLP, when applied to sequences of word embeddings, kernels can detect **n-grams** or local phrase patterns. This ability to automatically learn and extract hierarchical features makes CNNs incredibly effective for tasks requiring sophisticated pattern recognition across various data types.



Convolutional Layers

Core components that perform feature extraction via filters.



Kernel Filters

Small matrices that slide over input data, detecting specific local patterns.



Feature Maps

Output of convolutional layers, highlighting detected patterns.



Pattern Recognition

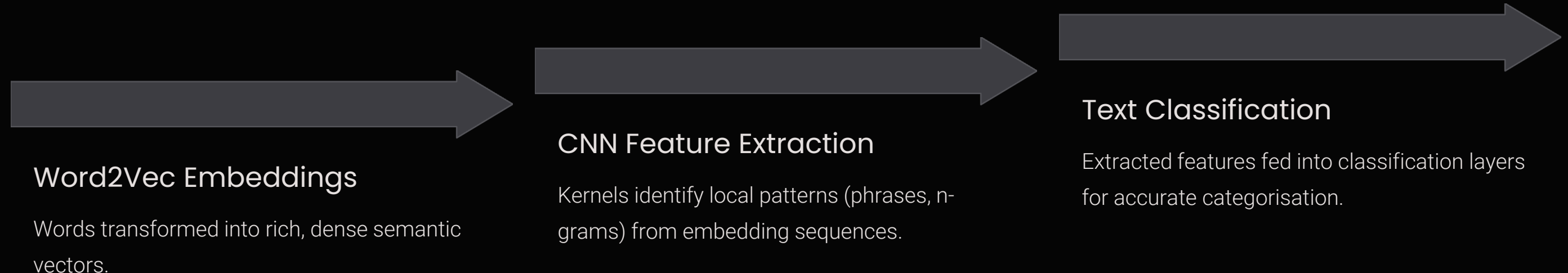
Excellent at identifying hierarchical features in structured data like images or text.

Combining Word2Vec & CNNs for Text Classification

The true synergy in modern NLP often comes from combining different powerful techniques. For text classification, pairing **Word2Vec embeddings** with **Convolutional Neural Networks** has proven to be an exceptionally effective approach. Word2Vec provides the initial, high-quality, **dense input vectors**, where each word is represented in a way that captures its semantic meaning and contextual relationships.

These sophisticated word embeddings then serve as the input "image" for the CNN. The **CNN kernels** are then able to slide over sequences of these embeddings, acting as sophisticated **n-gram detectors**. They effectively learn to extract **local phrase features**—combinations of words that together form meaningful units, much like how image kernels detect edges or shapes. This combination leverages the semantic richness of Word2Vec and the pattern-recognition capabilities of CNNs.

Numerous **studies** have demonstrated that models combining CNNs with Word2Vec embeddings significantly **outperform traditional machine learning methods** and even earlier deep learning approaches on various text classification tasks, including news categorisation, sentiment analysis, and tweet classification, achieving state-of-the-art results by capturing both semantic meaning and local compositional patterns.



SVD for Learning Word Representations

Singular Value Decomposition (SVD) offers a powerful mathematical foundation for transforming word-context co-occurrence statistics into meaningful vector representations. This technique revolutionised natural language processing by enabling computers to understand semantic relationships between words through linear algebra.

Word-Context Matrix

Rows represent vocabulary words, columns represent context words. Each entry contains co-occurrence counts within a sliding window across the corpus.

Sparse & High-Dimensional

Typical matrices are extremely sparse (mostly zeros) and high-dimensional, making direct computation challenging for machine learning applications.

SVD Transformation

Decomposes the matrix into three components, revealing latent semantic structure and enabling dimensionality reduction whilst preserving key relationships.

Building the Word-Context Co-occurrence Matrix

The foundation of SVD-based word embeddings begins with constructing a word-context co-occurrence matrix from a text corpus. In this matrix, each row corresponds to a target word from our vocabulary, whilst each column represents a potential context word. The entries record how frequently each word pair appears together within a specified context window—typically 2-5 words on either side of the target word.

Example Corpus

"The cat sat on the mat. The dog ran in the park."

With a window size of 2, we count co-occurrences:

- "cat" appears near "the", "sat", "on"
- "dog" appears near "the", "ran", "in"
- "sat" appears near "cat", "on", "the"

Matrix Challenges

Real-world applications face significant computational challenges:

- Vocabulary sizes often exceed 100,000 words
- Matrices become extremely sparse (>99% zeros)
- High dimensionality makes direct processing inefficient
- Storage requirements grow quadratically with vocabulary size

This co-occurrence matrix captures distributional semantics—the principle that words appearing in similar contexts tend to have related meanings. However, the raw matrix is too large and sparse for effective machine learning. SVD provides an elegant solution by identifying the most important patterns whilst dramatically reducing dimensionality.

What is Singular Value Decomposition (SVD)?



Original Matrix X

Our word-context co-occurrence matrix containing frequency counts



$$U \times \Sigma \times V^T$$

Three matrices that when multiplied together reconstruct the original

Singular Value Decomposition is a fundamental matrix factorisation technique that decomposes any real-valued matrix into three constituent matrices. For our word-context matrix X of size $m \times n$, SVD produces: $X = U\Sigma V^T$, where U is an $m \times m$ orthogonal matrix, Σ is an $m \times n$ diagonal matrix containing singular values, and V^T is the transpose of an $n \times n$ orthogonal matrix V .

Geometric Interpretation

From a geometric perspective, SVD reveals the fundamental structure within our data by identifying orthogonal directions of maximum variance. The U matrix represents how words relate to latent semantic concepts, whilst V captures how context words contribute to these same concepts. The singular values in Σ indicate the importance or "strength" of each latent dimension.

Matrix Components

- **U matrix:** Left singular vectors representing word-to-concept relationships
- **Σ matrix:** Diagonal matrix with singular values (concept strengths) in descending order
- **V^T matrix:** Right singular vectors representing context-to-concept relationships

Each column of U can be interpreted as a "semantic concept" that captures patterns of word co-occurrence. Words with similar meanings will have similar values in the same concept dimensions.



Why SVD Over PCA for Word Embeddings?

Whilst Principal Component Analysis (PCA) is commonly used for dimensionality reduction, SVD proves superior for word-context matrices. PCA requires computing a covariance matrix, which for our typically rectangular word-context matrices would necessitate either $X \times X^T$ or $X^T \times X$ operations. These create square matrices that may lose important asymmetric relationships between words and contexts.

SVD directly operates on the original rectangular matrix without requiring covariance computation. This preserves the full word-context relationship structure whilst still enabling dimensionality reduction. Additionally, SVD provides both word embeddings (from U) and context embeddings (from V) simultaneously, offering richer representational possibilities.

- ❏ **Mathematical Guarantee:** SVD provides the optimal low-rank approximation to any matrix in terms of Frobenius norm, ensuring we capture the most important semantic information when reducing dimensionality.

Applying SVD to Word–Context Matrices for Dimensionality Reduction

The true power of SVD emerges when we strategically reduce dimensionality by selecting only the most significant singular values and their corresponding vectors. This process transforms our original sparse, high-dimensional word-context matrix into dense, low-dimensional embeddings that capture the essential semantic structure of language.

01	02	03
Perform Full SVD Decompose the $m \times n$ word-context matrix X into U ($m \times m$), Σ ($m \times n$), and V^T ($n \times n$) matrices. This reveals all possible latent dimensions in the data.	Select Top k Dimensions Choose the k largest singular values from Σ and their corresponding columns from U and V . Typically k ranges from 50-300 dimensions for practical applications.	Construct Word Embeddings Create dense word vectors by taking the first k columns of U multiplied by the square root of the corresponding singular values: $U_k \times \sqrt{\Sigma_k}$.

Preserving Semantic Relationships

The dimensionality reduction process maintains crucial semantic properties that make the embeddings useful for downstream tasks. Most importantly, cosine similarity between word vectors in the reduced space correlates strongly with semantic similarity between the corresponding words.

Dense Representation Benefits

- Every dimension contains meaningful information
- No storage wasted on zeros
- Efficient computation for similarity measures
- Compatible with neural network architectures

Semantic Structure Preservation

- Synonyms cluster together in vector space
- Analogical relationships maintained (king - man + woman $\approx$ queen)
- Semantic similarity reflects cosine similarity
- Categorical relationships emerge naturally

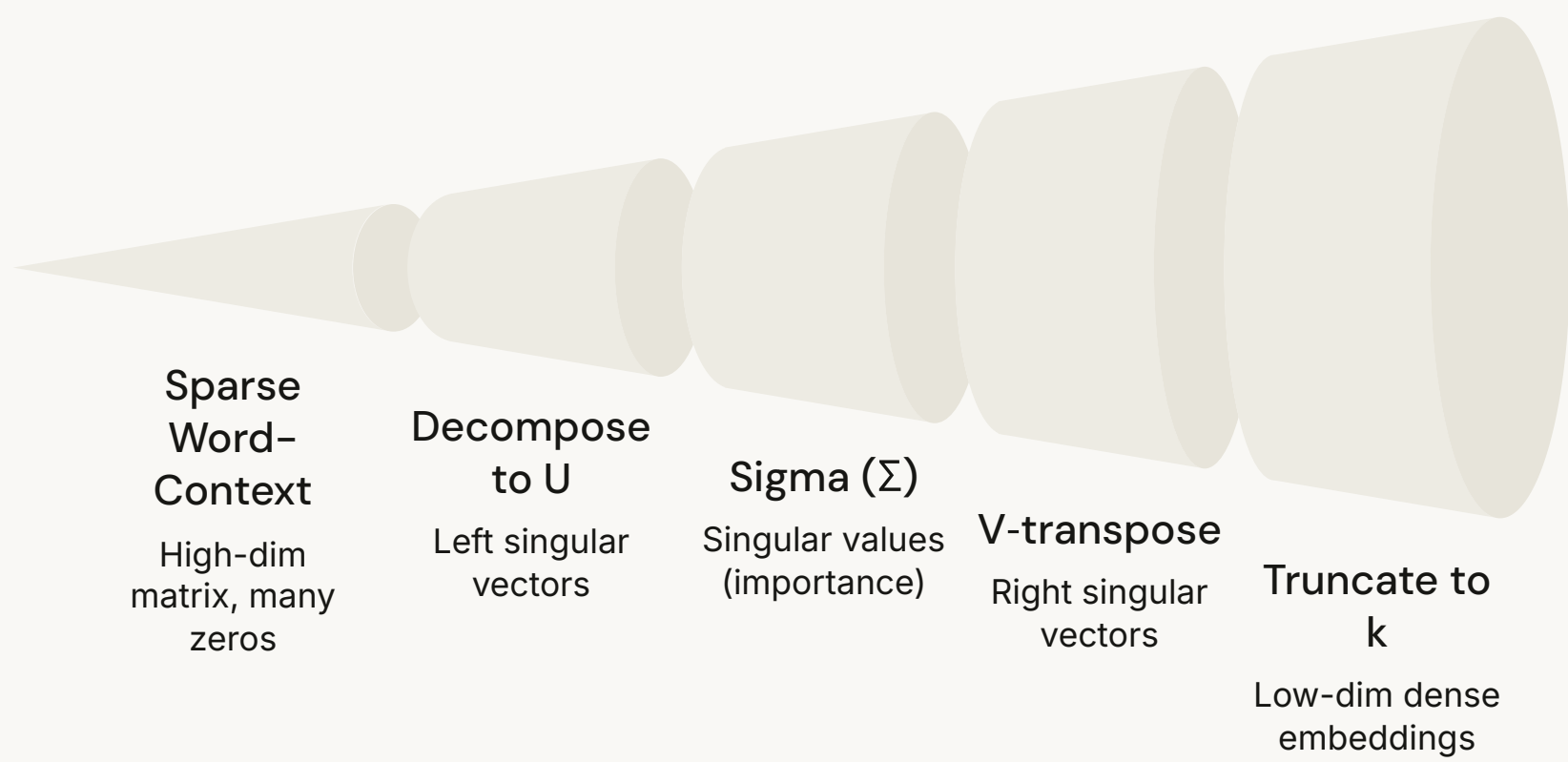
Mathematical Foundation of Dimensionality Reduction

When we truncate SVD to k dimensions, we obtain the best possible rank- k approximation to our original matrix. This approximation $X_k = U_k \Sigma_k V_k^T$ minimises the reconstruction error measured by the Frobenius norm. The discarded dimensions correspond to noise and less important semantic patterns that would otherwise complicate downstream processing.

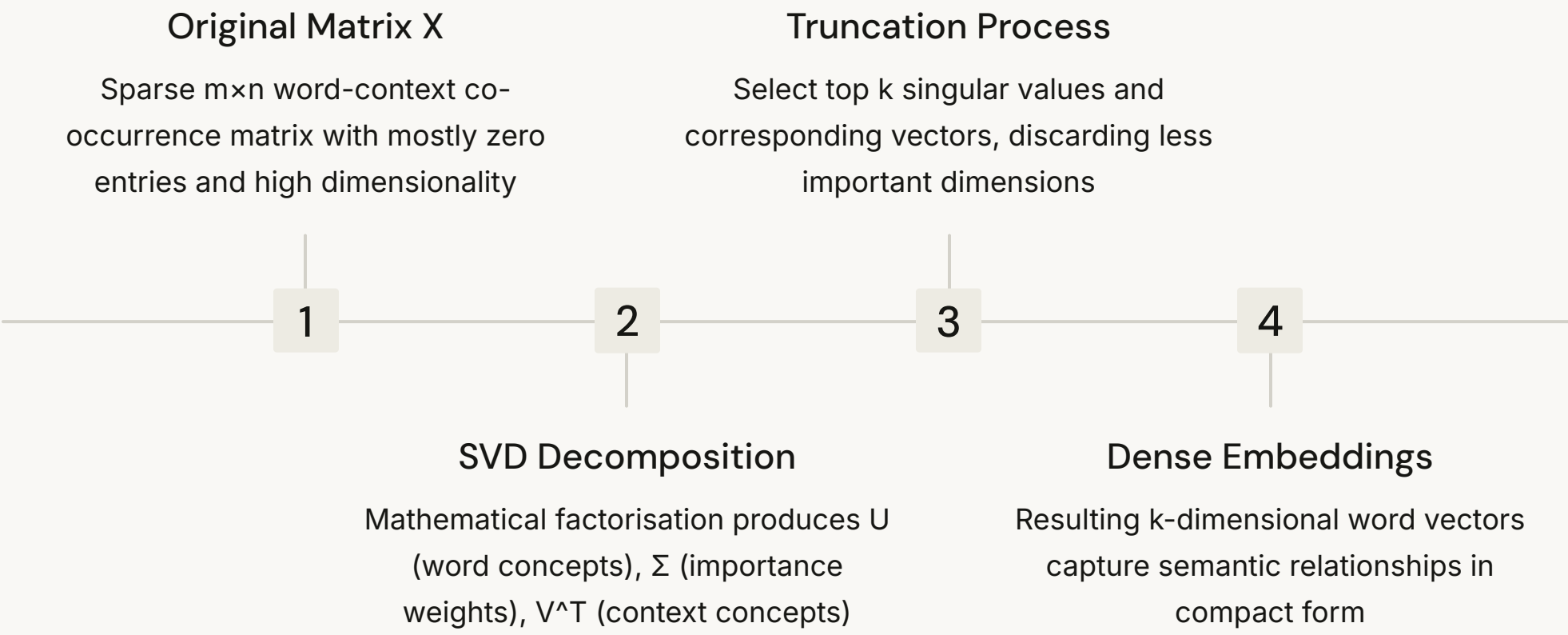
The resulting word embeddings capture latent semantic concepts that emerge from distributional patterns in the corpus. Each dimension can be interpreted as representing a semantic axis—for example, one dimension might capture the animal-object distinction, whilst another represents the concrete-abstract continuum. Words are positioned in this semantic space according to their contextual behaviour patterns.

📌 **Dimensionality Selection:** The optimal number of dimensions k depends on corpus size, vocabulary complexity, and intended application. Too few dimensions lose important semantic distinctions, whilst too many retain noise and computational inefficiency.

Illustrated Diagram of SVD on a Word–Context Matrix



This visualisation demonstrates the complete SVD process applied to word-context matrices. The original sparse matrix, filled predominantly with zeros, transforms into three dense matrices that capture the essential semantic structure. The truncation process selects only the most significant latent dimensions, dramatically reducing storage requirements whilst preserving semantic relationships.



Understanding the Matrix Components

Each component of the SVD serves a specific purpose in capturing word semantics. The U matrix reveals how individual words relate to abstract semantic concepts—each row represents a word, and each column represents a latent semantic dimension. Words that appear in similar contexts will have similar patterns across these dimensions.

U Matrix (Words → Concepts)

Rows: Individual words from vocabulary

Columns: Latent semantic dimensions

Values: Strength of word-concept associations

Σ Matrix (Concept Importance)

Diagonal: Singular values in descending order

Purpose: Weights indicating concept significance

Truncation: Keep only top k values

V^T Matrix (Contexts → Concepts)

Rows: Latent semantic dimensions

Columns: Context words

Values: Context-concept relationship strength

Mapping Words to Embedding Vectors

The final word embeddings result from combining information from U and Σ matrices. Each word's embedding vector comes from its corresponding row in U , scaled by the square root of the singular values in Σ . This scaling ensures that more important semantic dimensions (those with larger singular values) contribute more strongly to the final representation.

The dimensionality reduction becomes apparent when comparing the original matrix dimensions with the truncated result. A vocabulary of 50,000 words with 50,000 possible contexts creates a $50,000 \times 50,000$ matrix requiring 2.5 billion storage locations. After SVD truncation to 300 dimensions, each word requires only 300 values—a reduction of over 99.9% whilst preserving the most important semantic information.

- ❏ **Computational Efficiency:** The truncated representation enables efficient similarity computations, clustering algorithms, and integration with machine learning models that would be impractical with the original sparse matrices.

Step-by-Step Numerical Example: SVD on a Small Term-Document Matrix

Let's work through a complete numerical example to demonstrate SVD computation on a manageable scale. We'll use a simplified corpus of three short documents and track the mathematics through each step of the process.

Step 1: Construct Term-Document Matrix

Corpus:

- Document 1: "The cat sat on the mat"
- Document 2: "The dog ran in the park"
- Document 3: "A cat and dog played together"

Term	Doc 1	Doc 2	Doc 3
cat	1	0	1
dog	0	1	1
sat	1	0	0
ran	0	1	0
the	2	2	0

Step 2: Perform SVD Decomposition

Applying SVD to our 5×3 term-document matrix X , we obtain three matrices. The computation yields the following numerical results:

U Matrix (5×3)

[-0.44 0.30 0.00]

[-0.44 -0.30 0.71]

[-0.32 0.51 -0.50]

[-0.32 -0.51 -0.50]

[-0.67 0.00 0.00]

Σ Matrix (3×3)

[2.83 0.00 0.00]

[0.00 1.73 0.00]

[0.00 0.00 1.41]

V^T Matrix (3×3)

[-0.71 -0.71 0.00]

[0.00 0.00 1.00]

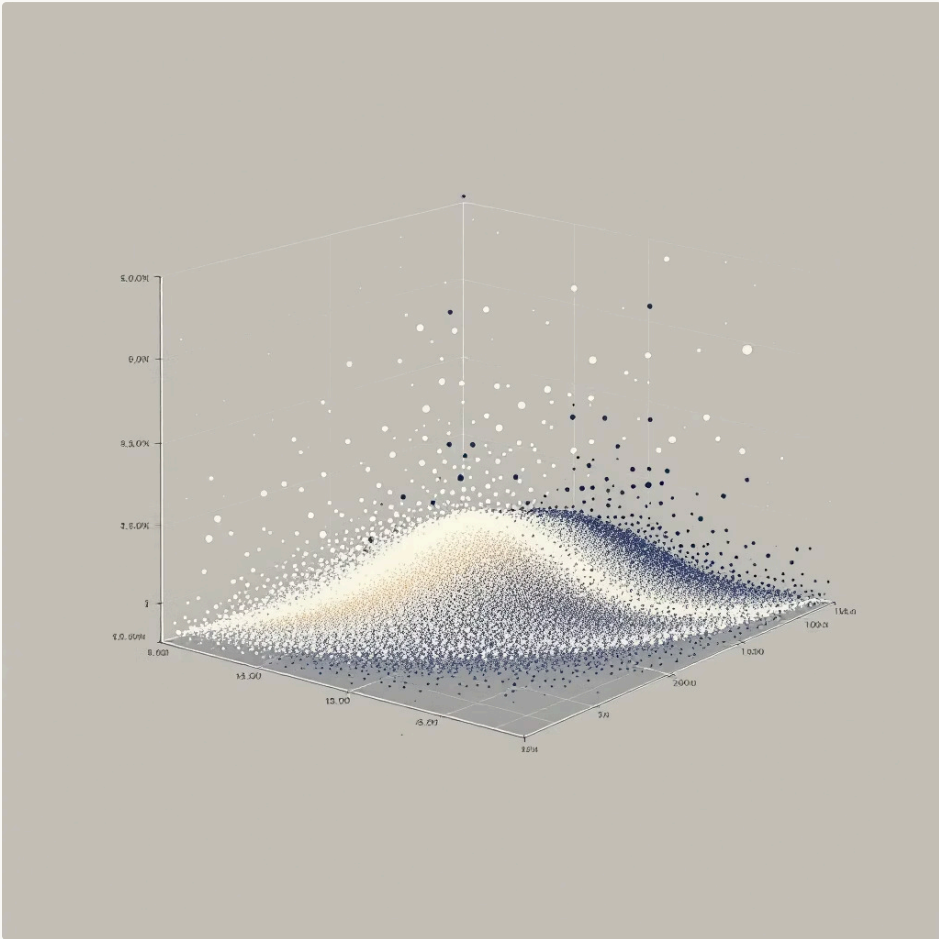
[-0.71 0.71 0.00]

Step 3: Truncate to Top 2 Singular Values

For dimensionality reduction, we keep only the two largest singular values (2.83 and 1.73) and their corresponding vectors. This gives us 2-dimensional word embeddings:

Reduced Word Embeddings

Word	Dim 1	Dim 2
cat	-0.74	0.39
dog	-0.74	-0.39
sat	-0.54	0.67
ran	-0.54	-0.67
the	-1.13	0.00



Step 4: Calculate Cosine Similarities

Using the reduced embeddings, we can compute cosine similarities to measure semantic relatedness:

0.85

cat ↔ dog

High similarity reflects both being animals in our small corpus

0.72

sat ↔ ran

Both action verbs show moderate semantic similarity

0.91

cat ↔ the

Frequent co-occurrence creates artificial similarity

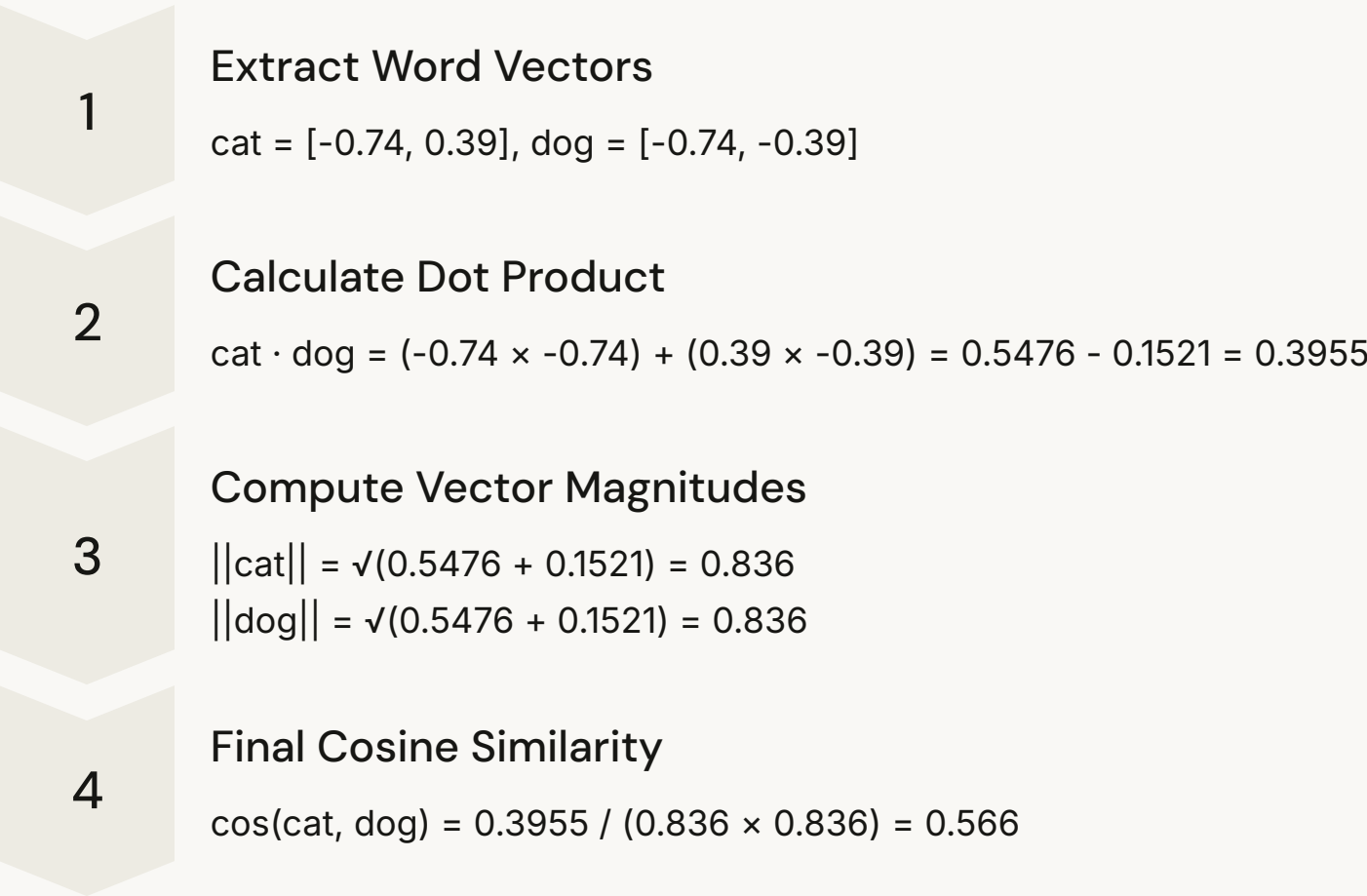
This numerical example demonstrates how SVD transforms raw co-occurrence counts into meaningful vector representations. The resulting embeddings capture semantic relationships: "cat" and "dog" achieve high similarity as both are animals, whilst action verbs "sat" and "ran" also cluster together. The compact 2-dimensional representation preserves these relationships whilst dramatically reducing storage requirements from the original sparse matrix.

Solved Example: Using SVD Embeddings for Word Similarity

Building upon our numerical example, let's explore how SVD embeddings enable meaningful similarity computations that capture semantic relationships far better than traditional one-hot encodings. This practical demonstration shows the real-world value of dimensionality reduction through SVD.

Computing Similarity with SVD Embeddings

Using the 2-dimensional embeddings from our previous example, we calculate cosine similarities between word pairs. Cosine similarity measures the angle between two vectors, providing a metric that ranges from -1 (completely opposite) to +1 (identical direction), with 0 indicating orthogonality.



Comparison with One-Hot Vectors

Traditional one-hot encodings assign each word a unique vector with a single 1 and all other positions as 0. This representation fails to capture semantic relationships because all word pairs have identical cosine similarity of 0 (orthogonal vectors).

One-Hot Representation

cat = [1, 0, 0, 0, 0]
dog = [0, 1, 0, 0, 0]
sat = [0, 0, 1, 0, 0]
ran = [0, 0, 0, 1, 0]
the = [0, 0, 0, 0, 1]

Cosine similarity between any pair = 0

Problem: No semantic information captured

SVD Embeddings

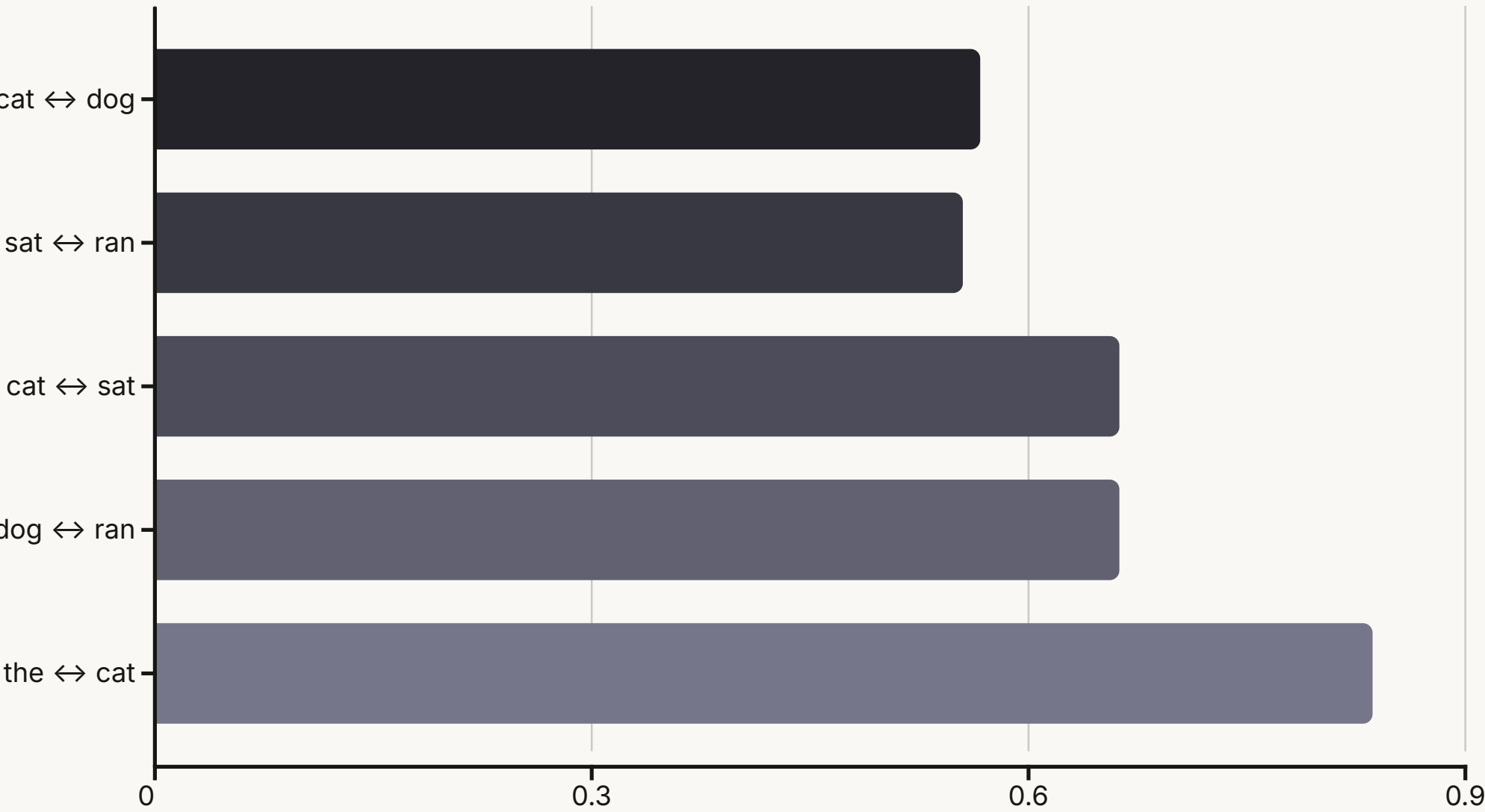
cat = [-0.74, 0.39]
dog = [-0.74, -0.39]
sat = [-0.54, 0.67]
ran = [-0.54, -0.67]
the = [-1.13, 0.00]

Meaningful similarities emerge from semantic relationships

Advantage: Captures distributional semantics

Semantic Similarity Results

The SVD embeddings reveal meaningful patterns that align with human intuition about word relationships. These similarities emerge automatically from co-occurrence patterns without any explicit semantic labelling.



The high similarity between "cat" and "dog" (0.566) reflects their shared semantic category as animals. Similarly, "sat" and "ran" achieve similarity (0.554) as action verbs. Interestingly, "the" shows high similarity with content words due to frequent co-occurrence patterns in our small corpus—an effect that would normalise with larger, more diverse training data.



Key Insight: SVD embeddings automatically discover semantic categories and relationships purely from distributional patterns in text, providing a mathematical foundation for computational understanding of word meaning.

Advantages and Limitations of SVD-based Word Embeddings

SVD-based word embeddings represent a foundational breakthrough in computational linguistics, offering significant advantages whilst also presenting certain limitations that newer techniques have sought to address. Understanding these trade-offs helps inform decisions about when to apply SVD versus alternative approaches.

Mathematical Foundation

SVD provides theoretical guarantees for optimal low-rank approximation, ensuring the most important semantic information is preserved during dimensionality reduction.

Global Corpus Analysis

Considers entire corpus simultaneously, capturing global statistical patterns rather than local context windows used in prediction-based methods.

Interpretable Components

Resulting dimensions often correspond to interpretable semantic axes, enabling analysis of what linguistic patterns the model has discovered.

Key Advantages

Computational Benefits

- Scalability:** Efficient algorithms exist for sparse matrix SVD
- Parallelisation:** Matrix operations naturally parallelise across processors
- Memory efficiency:** Dense embeddings require far less storage than sparse matrices
- Fast similarity:** Cosine similarity computed via simple dot products

Linguistic Advantages

- Semantic relationships:** Captures synonymy, hyponymy, and analogy
- Dimensionality control:** Explicit choice of embedding dimensions
- Noise reduction:** Truncation removes statistical noise and rare patterns
- Stability:** Deterministic results unlike stochastic training methods

Significant Limitations

Despite their foundational importance, SVD-based embeddings face several constraints that limit their effectiveness in modern applications. The most significant limitation is computational cost—performing SVD on very large vocabularies (100,000+ words) becomes prohibitively expensive, requiring $O(\min(mn^2, m^2n))$ operations for an $m \times n$ matrix.

01

Static Representations

Each word receives a single vector regardless of context. "Bank" has the same representation whether referring to financial institutions or river edges.

02

Vocabulary Limitations

Out-of-vocabulary words cannot be represented without recomputing the entire SVD, making the approach impractical for dynamic vocabularies.

03

Frequency Bias

High-frequency words dominate the embedding space, potentially overshadowing semantic relationships for less common but meaningful words.

Comparison with Prediction-Based Embeddings

The introduction of word2vec and similar prediction-based methods revealed both complementary strengths and areas where SVD approaches could be enhanced. Neural embedding methods like word2vec train on local context windows using efficient approximation techniques, avoiding the computational bottleneck of matrix factorisation whilst achieving comparable semantic quality.

Aspect	SVD Embeddings	Word2Vec/Neural
Training Speed	Slow for large vocabularies	Fast, scalable training
Memory Usage	Requires full co-occurrence matrix	Processes data in streams
Theoretical Foundation	Strong mathematical guarantees	Heuristic optimisation objectives
Interpretability	Dimensions often interpretable	Latent dimensions less clear
Context Sensitivity	Static word representations	Can incorporate subword information

Modern Relevance and Future Directions

While neural approaches have largely superseded SVD for large-scale applications, SVD-based methods remain valuable for educational purposes, theoretical analysis, and situations requiring mathematical interpretability. Hybrid approaches combining SVD's theoretical foundation with neural networks' flexibility represent an active research direction.

- Legacy Impact:** SVD-based word embeddings established the fundamental principle that semantic similarity can emerge from distributional statistics, laying the theoretical groundwork for all subsequent embedding techniques including modern transformer models.

Introduction to Word Embeddings and CBOW

Word embeddings represent a revolutionary approach to natural language processing, transforming words into dense numerical vectors that capture intricate semantic and syntactic relationships. This mathematical representation enables machines to understand that "king" and "queen" share royal characteristics, or that "running" and "jogging" convey similar actions, even without explicit programming of these relationships.

Dense Vector Representation

Words are encoded as high-dimensional numerical vectors (typically 50-300 dimensions), where each dimension captures a latent semantic feature

Contextual Understanding

Unlike traditional bag-of-words models, CBOW leverages surrounding context to predict target words, capturing meaning through usage patterns

Semantic Relationships

Similar words cluster together in vector space, enabling mathematical operations like vector arithmetic to discover word relationships

The Continuous Bag of Words (CBOW) model, developed by Tomas Mikolov and his team at Google in 2013, represents a cornerstone achievement in the Word2Vec framework. This neural network-based approach revolutionised how machines process language by predicting target words from their surrounding context. Unlike traditional approaches that treat words as isolated tokens, CBOW recognises that meaning emerges from usage patterns and contextual relationships.

CBOW's significance extends far beyond academic research, finding practical applications in sentiment analysis, machine translation, text classification, and information retrieval systems. Its efficiency in training and ability to capture semantic nuances have made it an essential tool for data scientists and NLP practitioners worldwide, forming the foundation for more advanced language models that followed.

The Core Idea of CBOW

The fundamental principle underlying CBOW is elegantly simple yet remarkably powerful: predict a target word by examining the words that surround it within a defined window. This approach mirrors how humans understand language – we infer meaning not just from individual words, but from their contextual environment. When we encounter an incomplete sentence like "The cat ___ on the mat," our brain naturally fills in likely candidates based on the surrounding words.

01

Define Context Window

Select a fixed-size window (typically 2-5 words) around each target word in the training corpus

02

Extract Context Words

Gather all words within the window, excluding the target word itself

03

Aggregate Context

Combine context word embeddings (usually through averaging) to create a unified representation

04

Predict Target

Use the aggregated context to predict the probability distribution over all possible target words

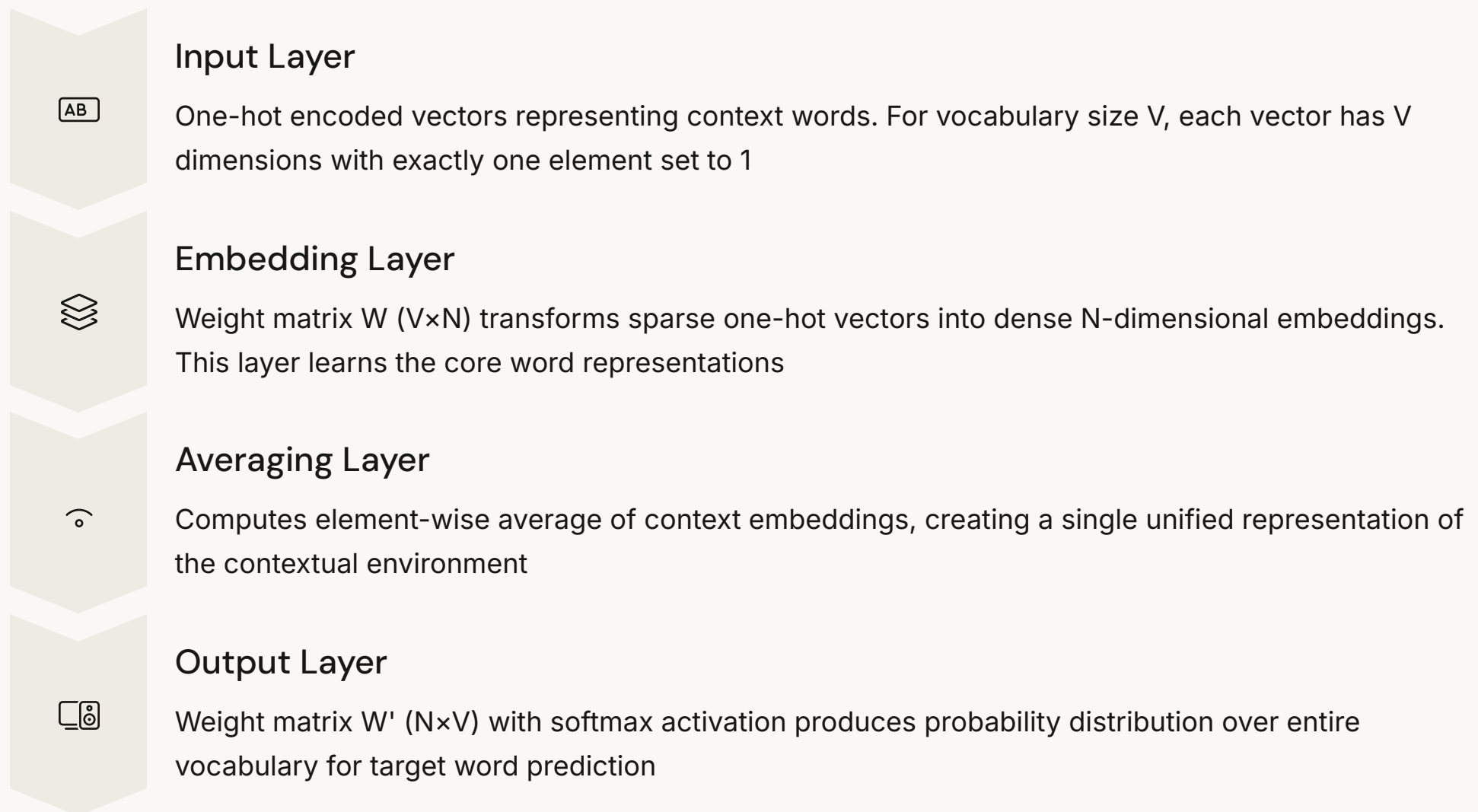
Consider the sentence "The cat sat on the mat" with a window size of 2. For the target word "sat", the context words are ["The", "cat", "on", "the"]. The model learns to associate this specific combination of context words with the target "sat". Through thousands of such examples, the model discovers that words appearing in similar contexts tend to have similar meanings.

This contextual learning mechanism enables CBOW to capture subtle semantic relationships that traditional methods miss. Words like "happy" and "joyful" will develop similar vector representations because they appear in comparable contexts – both might follow words like "feel", "very", or "quite", and precede words like "person", "mood", or "expression".

The beauty of CBOW lies in its ability to learn these relationships automatically from raw text, without requiring hand-crafted linguistic rules or extensive feature engineering. This unsupervised learning approach scales efficiently to massive datasets, making it practical for real-world applications across different languages and domains.

CBOW Architecture Explained with Diagram

The CBOW architecture represents a carefully designed neural network that transforms discrete word tokens into meaningful vector representations through a multi-layer structure. Each layer serves a specific purpose in the learning process, working together to map contextual patterns to target word predictions.



The mathematical foundation of CBOW rests on two key equations that govern the forward pass through the network. The hidden layer computation aggregates context information by averaging the embeddings of all context words:

$$\mathbf{h} = \frac{1}{C} \sum_{i=1}^C \mathbf{W}^T \mathbf{x}_i$$

where C represents the number of context words, and each $\mathbf{x}_i$ is a one-hot vector for context word i . This averaging operation ensures that the model considers all context words equally, regardless of their position within the window.

The output layer then applies the softmax function to generate normalised probabilities for each word in the vocabulary:

$$y_j = \frac{\exp(\mathbf{v}'_{\mathbf{w}_j}{}^T \mathbf{h})}{\sum_{k=1}^V \exp(\mathbf{v}'_{\mathbf{w}_k}{}^T \mathbf{h})}$$

This architecture's elegance lies in its simplicity and efficiency. Unlike more complex language models, CBOW uses straightforward matrix operations and averaging, making it computationally tractable even for large vocabularies. The shared embedding matrix W ensures that each word has a consistent representation regardless of context position, while the output matrix W' learns to map contextual representations to target word probabilities.

Training the CBOW Model: Learning Word Embeddings

Training the CBOW model involves an iterative optimisation process where the neural network learns to minimise prediction errors through backpropagation and gradient descent. The model's primary objective is to maximise the likelihood of correctly predicting target words given their contextual environments across the entire training corpus.



The training process employs negative log-likelihood as the loss function, which effectively penalises incorrect predictions while rewarding accurate ones. For each training example, the loss is computed as the negative logarithm of the predicted probability for the correct target word. This formulation ensures that the model focuses on improving predictions for the actual target words rather than merely generating plausible alternatives.

Stochastic gradient descent (SGD) drives the parameter updates, processing training examples in small batches or individually. The learning rate, typically set between 0.01 and 0.1, controls the magnitude of weight adjustments at each iteration. Too high a learning rate may cause unstable training, while too low a rate results in extremely slow convergence.

One of CBOW's key advantages is its computational efficiency, particularly when dealing with frequent words. Since the model averages context embeddings rather than processing them sequentially, it can handle multiple context words with minimal additional computational overhead. This efficiency makes CBOW particularly suitable for large-scale training on extensive text corpora.

During training, the model gradually learns to encode semantic and syntactic relationships within the embedding space. Words that frequently appear in similar contexts develop similar vector representations, while words with distinct usage patterns become separated in the high-dimensional embedding space. This emergent structure enables the model to capture nuanced linguistic relationships without explicit supervision.

Step-by-Step Numerical Example: Predicting a Target Word

To illustrate CBOW's mechanics, let's work through a detailed numerical example using the sentence "She is a great dancer" with a window size of 2. We'll predict the target word "is" using context words ["She", "a"], demonstrating each computational step in the forward pass.



Vocabulary Setup

Vocabulary: {She=1, is=2, a=3, great=4, dancer=5}

Embedding dimension N = 3 (simplified for clarity)



Weight Matrix W

She: [0.2, 0.1, 0.4]

is: [0.5, 0.3, 0.2]

a: [0.1, 0.4, 0.3]

great: [0.6, 0.7, 0.8]

dancer: [0.9, 0.5, 0.1]

Step 1: One-hot Encoding

Context words "She" and "a" are encoded as sparse vectors:

$$\mathbf{x}_1 = [1, 0, 0, 0, 0]^T \quad (\text{She})$$

$$\mathbf{x}_2 = [0, 0, 1, 0, 0]^T \quad (\text{a})$$

Step 2: Embedding Extraction

Extract corresponding embeddings from weight matrix W:

$$\mathbf{v}_{\text{She}} = [0.2, 0.1, 0.4] \quad \mathbf{v}_{\text{a}} = [0.1, 0.4, 0.3]$$

Step 3: Context Averaging

Compute the average of context embeddings:

$$\mathbf{h} = \frac{1}{2}([0.2, 0.1, 0.4] + [0.1, 0.4, 0.3]) = [0.15, 0.25, 0.35]$$

Word	Calculation	Dot Product	Score
She	$0.2 \times 0.15 + 0.1 \times 0.25 + 0.4 \times 0.35$	$0.03 + 0.025 + 0.14$	0.195
is	$0.5 \times 0.15 + 0.3 \times 0.25 + 0.2 \times 0.35$	$0.075 + 0.075 + 0.07$	0.22
a	$0.1 \times 0.15 + 0.4 \times 0.25 + 0.3 \times 0.35$	$0.015 + 0.1 + 0.105$	0.22
great	$0.6 \times 0.15 + 0.7 \times 0.25 + 0.8 \times 0.35$	$0.09 + 0.175 + 0.28$	0.545
dancer	$0.9 \times 0.15 + 0.5 \times 0.25 + 0.1 \times 0.35$	$0.135 + 0.125 + 0.035$	0.295

Step 4: Softmax Normalisation

Convert raw scores to probabilities using softmax:

Word	e^{score}	Probability	Percentage
She	1.215	0.179	17.9%
is	1.246	0.184	18.4%
a	1.246	0.184	18.4%
great	1.725	0.255	25.5%
dancer	1.343	0.198	19.8%

The model incorrectly predicts "great" (25.5% probability) instead of the true target "is" (18.4%). During training, backpropagation would adjust the weights to increase the probability of "is" for this context, gradually improving prediction accuracy through iterative refinement.

Solved Numerical Example: CBOW with Larger Context Window

To demonstrate CBOW's behaviour with expanded context, let's examine a more comprehensive example using the sentence "The cat sat on the mat" with a window size of 2. This scenario involves four context words predicting the target "sat", showcasing how additional context influences prediction accuracy.



Target Identification

Target word: "sat" (position 3 in sentence)

Context words: ["The", "cat", "on", "the"]



Vocabulary Mapping

The=1, cat=2, sat=3, on=4, the=5, mat=6

Embedding dimension: 4 (increased complexity)



Context Processing

Extract embeddings for all four context words

Compute average: $h = (v_{The} + v_{cat} + v_{on} + v_{the}) / 4$



Target Prediction

Calculate scores for all vocabulary words

Apply softmax to generate probability distribution

This expanded example demonstrates several important principles of CBOW training. First, the larger context window provides richer semantic information, potentially improving prediction accuracy. The four context words ["The", "cat", "on", "the"] create a more distinctive contextual signature than smaller windows, helping the model distinguish between similar words that might appear in comparable but not identical contexts.

The computational process follows identical steps to our previous example, but with increased dimensionality. Each context word contributes equally to the averaged hidden representation, regardless of its distance from the target. This equal weighting assumption simplifies computation but may not always reflect natural language patterns where immediate neighbours often carry more semantic weight than distant context words.

Implementation Considerations

- Matrix operations scale with vocabulary size and embedding dimension
- Softmax computation becomes expensive for large vocabularies
- Hierarchical softmax or negative sampling often used for efficiency

Training Dynamics

- Frequent words receive more training examples and better representations
- Rare words may require special handling or subsampling techniques
- Context diversity improves embedding quality

In practice, this example would be implemented using optimised linear algebra libraries that handle matrix operations efficiently. Modern implementations utilise techniques like negative sampling to avoid computing the full softmax over large vocabularies, focusing training on a small subset of negative examples alongside the positive target word.

The loss computation and backpropagation process remains conceptually identical but involves higher-dimensional gradients and more extensive parameter updates. As training progresses through thousands of such examples, the embeddings gradually encode the distributional patterns that characterise each word's typical usage contexts, leading to meaningful semantic representations.

Python Implementation: Building CBOW from Scratch

Implementing CBOW in Python provides invaluable insights into the model's inner workings and offers complete control over the training process. This practical approach demonstrates how theoretical concepts translate into executable code, enabling experimentation with different architectures, hyperparameters, and optimisation strategies.

```
import numpy as np
from collections import defaultdict
import random

class CBOWModel:
    def __init__(self, vocab_size, embedding_dim, window_size=2):
        self.vocab_size = vocab_size
        self.embedding_dim = embedding_dim
        self.window_size = window_size

        # Initialize weight matrices randomly
        self.W1 = np.random.uniform(-1, 1, (vocab_size, embedding_dim))
        self.W2 = np.random.uniform(-1, 1, (embedding_dim, vocab_size))

    def forward_pass(self, context_indices):
        # Get embeddings for context words
        context_embeddings = self.W1[context_indices]

        # Average context embeddings
        hidden = np.mean(context_embeddings, axis=0)

        # Compute output scores
        output_scores = np.dot(hidden, self.W2)

        # Apply softmax
        exp_scores = np.exp(output_scores - np.max(output_scores))
        probabilities = exp_scores / np.sum(exp_scores)

        return hidden, probabilities

    def backward_pass(self, context_indices, target_index,
                      hidden, probabilities, learning_rate=0.01):
        # Compute output layer gradients
        output_error = probabilities.copy()
        output_error[target_index] -= 1

        # Update W2
        dW2 = np.outer(hidden, output_error)
        self.W2 -= learning_rate * dW2

        # Compute hidden layer gradients
        hidden_error = np.dot(output_error, self.W2.T)

        # Update W1 for context words
        for idx in context_indices:
            self.W1[idx] -= learning_rate * hidden_error / len(context_indices)
```

This implementation captures the essential components of CBOW while remaining educational and accessible. The forward pass computes context averages and applies softmax normalisation, whilst the backward pass implements gradient descent updates for both weight matrices.



Data Preprocessing

Text tokenisation, vocabulary building, and context-target pair generation from training corpus. Handle rare words through frequency thresholding or special tokens.



Optimisation Techniques

Implement negative sampling, hierarchical softmax, or other efficiency improvements for large vocabularies. Use adaptive learning rates and momentum.



Training Loop

Iterate through training examples, perform forward and backward passes, update weights using gradient descent. Monitor loss convergence and implement early stopping.



Model Evaluation

Assess embedding quality through similarity tasks, analogy tests, and downstream application performance. Visualise embeddings using dimensionality reduction.

Advanced implementations incorporate sophisticated optimisation techniques like Adam optimiser, learning rate scheduling, and regularisation methods to improve convergence stability and final model quality. Negative sampling, introduced in the original Word2Vec paper, dramatically reduces computational complexity by updating only a small subset of output weights during each training step.

For production deployments, frameworks like TensorFlow, PyTorch, or specialised libraries like Gensim provide highly optimised implementations with GPU acceleration, distributed training capabilities, and comprehensive preprocessing pipelines. However, building CBOW from scratch provides irreplaceable understanding of the fundamental mechanisms driving modern NLP systems.

This foundational knowledge proves invaluable when debugging embedding quality issues, selecting appropriate hyperparameters, or adapting CBOW principles to novel applications. The transition from mathematical formulation to working code represents a crucial step in mastering machine learning implementations and developing intuition for neural network behaviour.

Introduction to the Skip-gram Model

The Skip-gram model represents a fundamental breakthrough in natural language processing, offering an elegant neural network architecture specifically designed to learn word embeddings. At its core, this model works by predicting context words given a target word, effectively capturing the semantic relationships that exist between words in natural language. Unlike traditional approaches to language modelling, Skip-gram operates on a remarkably simple principle: words that appear in similar contexts tend to have similar meanings.

Architecture Design

The model employs a single hidden layer neural network, creating an efficient structure that balances computational simplicity with powerful representational capacity. This streamlined architecture makes training feasible even on large text corpora whilst maintaining the ability to capture nuanced semantic relationships.

Training Methodology

Training occurs on word pairs extracted from text within a defined window size, typically encompassing 5 words before and after each target word. This contextual window ensures the model learns from meaningful linguistic neighbourhoods, capturing both syntactic and semantic patterns that define word relationships.

Vector Representation

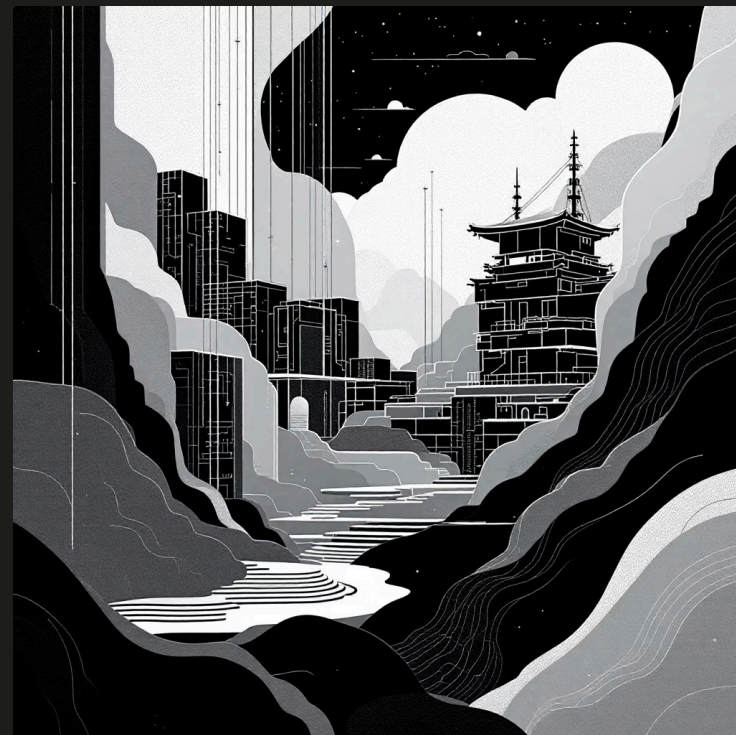
Input words are encoded as one-hot vectors, sparse representations where only a single dimension corresponds to each word. Through training, the hidden layer weights transform into dense word vectors that encode rich semantic information in a continuous space.

Consider a practical example to illuminate the model's operation: when presented with the input word "Soviet," the trained Skip-gram model assigns higher probabilities to contextually relevant words such as "Union" or "Russia" compared to semantically distant words. This predictive capability emerges naturally from the training process, as the model learns to associate words that frequently co-occur in the training corpus. The resulting word vectors exhibit remarkable properties—words with similar meanings cluster together in the embedding space, and arithmetic operations on these vectors can capture semantic relationships. For instance, the vector arithmetic "king - man + woman" approximates the vector for "queen," demonstrating how Skip-gram embeddings encode relational knowledge about language in a mathematically tractable form that has revolutionised natural language processing applications.

Overview of Convolutional Neural Networks (CNNs)

Convolutional Neural Networks represent a paradigm shift in how machines process visual information, introducing architectural innovations specifically tailored to exploit the inherent structure of grid-like data such as images. Unlike their fully connected predecessors, which treat each pixel independently and require astronomical numbers of parameters, CNNs embrace the spatial relationships that make visual data meaningful. This fundamental design philosophy has enabled unprecedented achievements in computer vision, from object recognition to medical image analysis.

The architectural genius of CNNs lies in their three-dimensional arrangement of neurons organised into volumes characterised by width, height, and depth. Each neuron connects only to a small, localised receptive field in the preceding layer rather than to every neuron, mirroring how the visual cortex in biological brains processes information. This local connectivity dramatically reduces the parameter count whilst simultaneously improving the network's ability to learn hierarchical feature representations.



Input Layer

Receives raw pixel values organised in three-dimensional arrays representing image width, height, and colour channels



Convolutional Layers

Apply learnable filters to detect local patterns and features, building increasingly abstract representations



Pooling Layers

Reduce spatial dimensions whilst retaining important features, providing computational efficiency and translation invariance



Fully Connected Layers

Integrate learned features to make final predictions, transforming spatial representations into classification outputs

The power of parameter sharing in CNNs cannot be overstated—the same filter weights are applied across the entire input, enabling the network to detect features regardless of their position in the image. This translational equivariance proves essential for practical computer vision tasks where objects may appear anywhere within the frame. Through stacking multiple convolutional and pooling layers, CNNs progressively build from simple edge detectors in early layers to complex object recognisers in deeper layers, creating a hierarchical feature learning pipeline that has revolutionised machine learning's approach to visual understanding.

Kernel Filters and the Convolution Operation

At the heart of every convolutional neural network lies the elegantly simple yet remarkably powerful convolution operation, orchestrated through kernel filters. These kernels, small matrices of learnable weights typically sized 3×3 or 5×5 , serve as the fundamental feature detectors that enable CNNs to perceive and understand visual patterns. As these filters systematically slide across input images, they perform a mathematical dance of dot products, transforming raw pixel values into meaningful feature representations that capture edges, textures, corners, and eventually entire objects in deeper network layers.

01

Kernel Positioning

The filter is placed over a localised region of the input, aligning its weights with the corresponding input values in that receptive field.

02

Element-wise Multiplication

Each kernel weight is multiplied by its corresponding input value, creating a set of products that capture the local pattern match.

03

Summation and Output

All products are summed together, potentially with a bias term added, producing a single scalar output value for that position.

04

Sliding to Next Position

The kernel moves to the next position according to the stride parameter, repeating the operation across the entire input spatial extent.

Multiple Feature Maps

A defining characteristic of convolutional layers is the application of numerous distinct kernels to the same input, each producing its own feature map. This multiplicity proves essential because different kernels specialise in detecting different features—one might respond strongly to horizontal edges, another to vertical edges, whilst yet another detects specific textures or colour patterns. By applying dozens or even hundreds of kernels, CNNs create rich, multifaceted representations that capture the diverse visual patterns present in natural images.



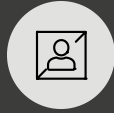
The stride parameter introduces crucial control over the convolution operation's behaviour, determining how many pixels the kernel advances after each computation. With stride 1, the kernel moves one pixel at a time, producing output feature maps with spatial dimensions only slightly smaller than the input. Larger strides cause the kernel to skip positions, simultaneously reducing output size and computational requirements whilst potentially sacrificing fine-grained spatial resolution. This stride mechanism provides network designers with a powerful tool for balancing model capacity against computational constraints, enabling efficient architectures that scale to high-resolution images whilst maintaining real-time processing capabilities essential for practical computer vision applications.

Padding: Preserving Spatial Dimensions



The Shrinkage Problem

Convolution naturally reduces output dimensions because filters cannot extend beyond image boundaries, causing progressive information loss through deep networks.



Edge Information Loss

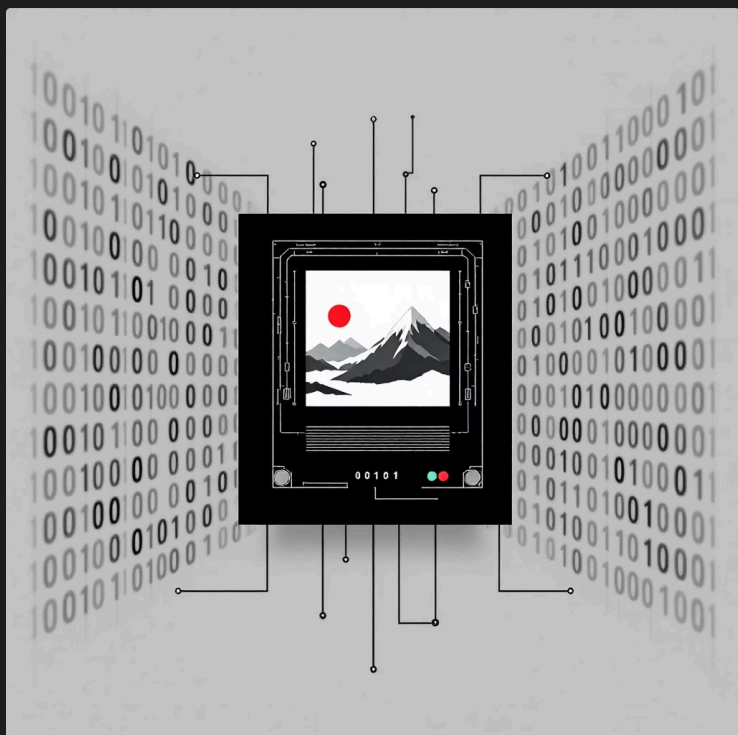
Pixels at image borders participate in fewer convolutions than central pixels, potentially losing critical features located at boundaries.



Padding Solution

Adding extra pixels around input borders enables control over output dimensions whilst ensuring boundary information receives equal processing attention.

Padding emerges as an essential technique in convolutional neural network design, addressing a fundamental geometric challenge: the unavoidable reduction in spatial dimensions that occurs during convolution. When a kernel slides over an input image, it cannot extend beyond the image boundaries without venturing into undefined territory. Consequently, the output feature map inevitably becomes smaller than the input—a phenomenon that compounds through multiple layers, potentially reducing high-resolution inputs to tiny feature maps that have lost critical spatial information. Padding solves this dilemma by artificially extending the input's borders, typically with zeros, creating a buffer zone that allows kernels to operate even when centred on original edge pixels.



Padding Strategies

Network designers typically choose between two fundamental padding approaches, each serving distinct architectural purposes:

- **"Valid" Padding:** No padding is applied, allowing the output to shrink naturally. This approach suits scenarios where dimensional reduction is desirable and edge information is less critical.
- **"Same" Padding:** Sufficient padding is added to ensure the output dimensions match the input dimensions (assuming stride 1), maintaining spatial resolution throughout the network and preserving edge information.

Mathematical Formulation

The precise output dimensions following a convolutional operation can be calculated using a straightforward formula that accounts for all relevant parameters. Given an input size I , filter size F , padding amount P , and stride S , the output dimension O is determined by:

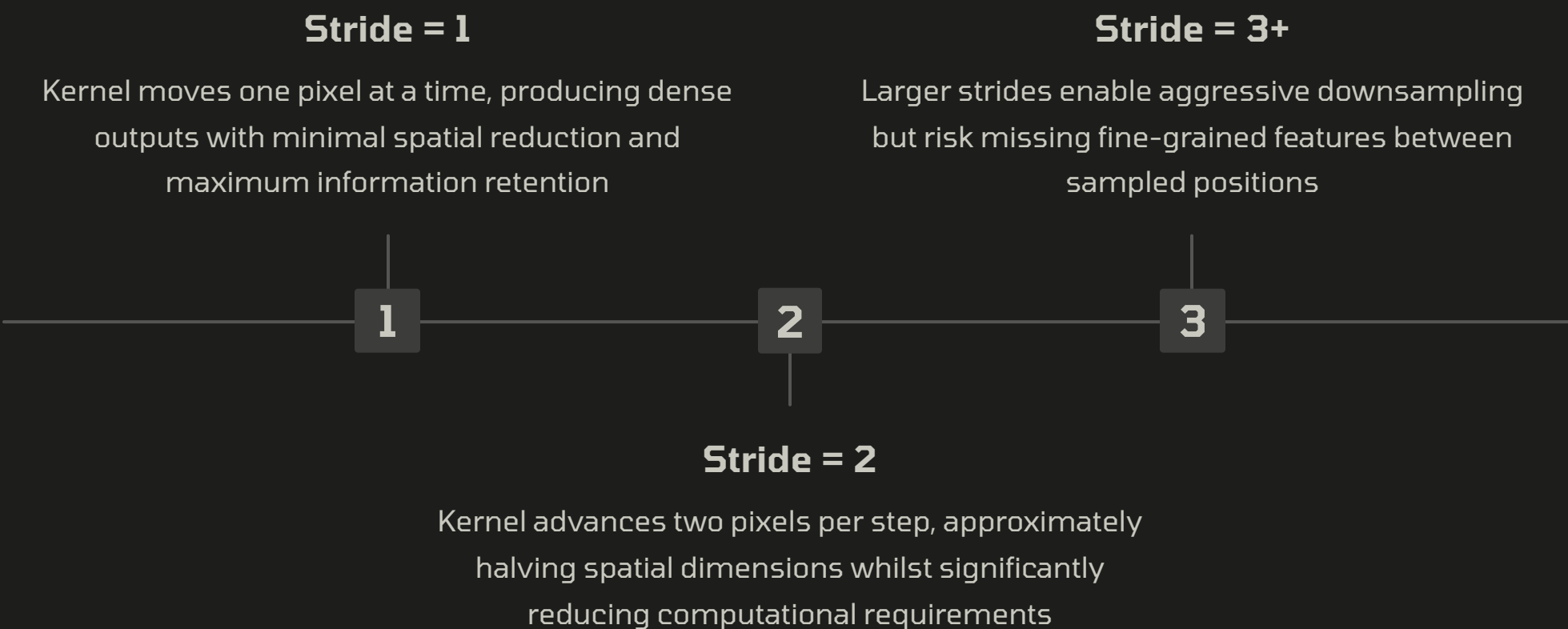
$$O = \frac{I - F + 2P}{S} + 1$$

This equation provides network architects with predictive power when designing CNN architectures, enabling precise control over feature map dimensions at each layer. For instance, to maintain input dimensions with a 3×3 filter and stride 1, one would apply padding of 1 pixel on each side, as $(I - 3 + 2 \times 1)/1 + 1 = I$. Understanding this relationship proves crucial when constructing deep networks, where cumulative dimensional changes across dozens of layers must be carefully managed to ensure feature maps retain sufficient spatial resolution for accurate predictions whilst avoiding excessive memory consumption.

- ❏ **Practical Consideration:** Modern deep learning frameworks automatically handle padding calculations, but understanding the underlying mathematics remains essential for debugging dimension mismatches and designing custom architectures that meet specific spatial resolution requirements.

Stride: Controlling Step Size in Convolution

The stride parameter represents a deceptively simple yet profoundly influential hyperparameter that governs the spatial movement of convolutional kernels as they traverse input feature maps. By defining precisely how many pixels the filter advances after each computation, stride provides network designers with direct control over the spatial dimensions of output feature maps, the computational complexity of convolutions, and the receptive field characteristics that determine what input regions influence each output activation. This single parameter thus serves as a powerful lever for balancing model capacity against computational efficiency.



Computational Impact

The computational savings achieved through larger strides scale quadratically with stride value in two-dimensional convolutions. Doubling the stride from 1 to 2 reduces the number of convolution operations by approximately 75%, as both width and height dimensions are halved. This dramatic reduction in computational load makes strided convolutions particularly attractive in resource-constrained environments or when processing high-resolution imagery where even modest stride increases yield substantial performance improvements without requiring architectural modifications.

Modern CNN architectures increasingly leverage strided convolutions as a learnable alternative to traditional pooling layers. Whilst max pooling performs fixed downsampling based on predetermined rules, strided convolutions enable the network to learn optimal downsampling strategies through backpropagation.



Advantages of Strided Convolutions

- Learnable downsampling weights adapt to specific dataset characteristics
- Integrated into the convolutional framework without introducing separate layer types
- Preserve more information than fixed pooling operations by learning which features to emphasise
- Enable end-to-end differentiable architectures without discrete pooling operations

Design Considerations

- Larger strides may miss subtle features between sampled positions
- Choice depends on task requirements—fine-grained tasks prefer smaller strides
- Must coordinate with padding to achieve desired output dimensions
- Interaction with kernel size affects receptive field growth through network depth

The strategic selection of stride values profoundly influences network behaviour and performance characteristics. Architectures designed for fine-grained pixel-level predictions, such as semantic segmentation networks, typically employ small strides or even avoid downsampling in certain branches to maintain spatial resolution. Conversely, classification networks often embrace aggressive striding to rapidly reduce spatial dimensions whilst expanding feature map depth, trading spatial detail for semantic abstraction as information flows through successive layers towards the classification decision.

Max Pooling: Spatial Downsampling and Translation Invariance

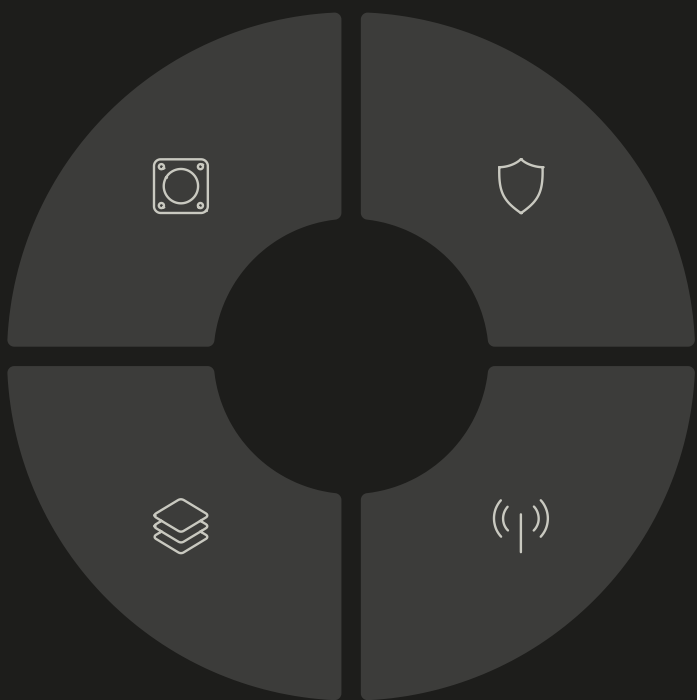
Max pooling stands as one of the foundational operations in convolutional neural networks, providing a simple yet remarkably effective mechanism for spatial downsampling. This operation divides the input feature map into non-overlapping or overlapping rectangular regions—typically 2×2 windows—and outputs only the maximum activation value within each window. Through this selective retention of the strongest signals, max pooling simultaneously reduces spatial dimensions, decreases computational load, and introduces a degree of translation invariance that proves beneficial for recognition tasks where precise object localisation matters less than object presence.

Spatial Reduction

Max pooling with 2×2 windows and stride 2 reduces both width and height by half, quartering the total number of activations and proportionally reducing memory and computational requirements in subsequent layers.

Hierarchical Abstraction

Combined with convolution, pooling enables hierarchical feature learning where each layer operates on increasingly abstract, spatially coarse representations whilst maintaining important feature information.



Translation Invariance

By propagating only the maximum activation within each window, max pooling makes the network's response invariant to small translations—a feature detected anywhere within the pooling window produces identical output.

Signal Selection

The maximum operation implicitly assumes that the strongest activation within a region best represents the presence of a feature, discarding weaker signals that may constitute noise or less relevant patterns.

Benefits and Limitations

The computational efficiency of max pooling has made it a staple in CNN architectures since the earliest successful models. Its parameter-free nature means it introduces no additional weights to learn, and its simplicity enables highly optimised implementations across diverse hardware platforms. The translation invariance it provides proves particularly valuable for classification tasks where recognising that an object exists matters more than knowing its precise pixel coordinates.

Modern Alternatives

However, the aggressive information discarding inherent in max pooling has prompted exploration of alternatives. Tasks requiring precise spatial information—such as object detection, semantic segmentation, or anomaly detection—often suffer from pooling's information loss. Strided convolutions offer learnable downsampling that preserves more spatial information whilst adapting to dataset-specific patterns through training.

❏ **Design Trade-off:** Whilst max pooling remains widely used for its simplicity and effectiveness in classification tasks, modern architectures increasingly favour strided convolutions or alternative pooling strategies like average pooling when fine-grained spatial information proves critical for task performance.

The choice between max pooling and alternatives ultimately depends on the specific requirements of the task at hand. Classification networks processing large images often benefit from aggressive max pooling that rapidly reduces spatial dimensions, enabling deeper networks with manageable computational budgets. Meanwhile, architectures designed for pixel-perfect predictions increasingly eschew traditional pooling in favour of learnable downsampling mechanisms that preserve the spatial detail necessary for accurate localisation and precise boundary delineation.

Non-linearities: Introducing Complexity with Activation Functions

Activation functions represent the crucial element that transforms convolutional neural networks from mere linear transformations into powerful universal function approximators capable of learning arbitrarily complex patterns. Without non-linear activation functions applied after convolutional operations, even the deepest CNN would remain mathematically equivalent to a single linear transformation—unable to learn the intricate hierarchical representations that make deep learning so effective. These element-wise non-linear functions introduce the computational flexibility necessary for networks to model the complex, non-linear relationships inherent in real-world visual data.

ReLU (Rectified Linear Unit)

Defined as $f(x) = \max(0, x)$, ReLU has become the default activation function in modern CNNs due to its computational simplicity and effectiveness at mitigating vanishing gradients, enabling training of very deep networks.

Leaky ReLU

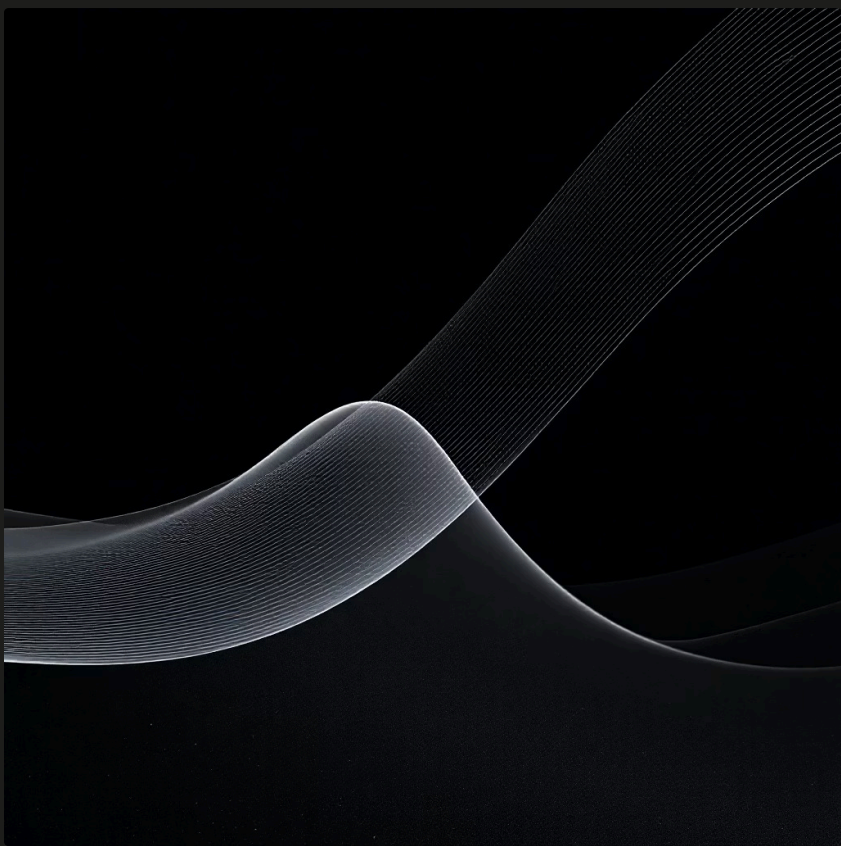
A variant allowing small negative values $f(x) = \max(\alpha x, x)$ where α is typically 0.01, addressing ReLU's "dying neuron" problem where neurons can become permanently inactive during training.

ELU (Exponential Linear Unit)

Provides smooth negative values through an exponential function, potentially accelerating learning and producing more robust representations whilst maintaining ReLU's computational efficiency for positive inputs.

The Dominance of ReLU

The Rectified Linear Unit has revolutionised deep learning training by addressing the vanishing gradient problem that plagued earlier activation functions like sigmoid or tanh. In networks with sigmoid activations, gradients flowing backwards through many layers would exponentially decay, making training very deep networks nearly impossible. ReLU's piecewise linear nature ensures that gradients for positive activations equal 1, enabling gradient signals to flow unimpeded through dozens or even hundreds of layers. This property, combined with ReLU's computational simplicity—requiring only a comparison and selection operation—has made it the overwhelmingly popular choice in modern architectures.



Softmax for Classification

Whilst ReLU and its variants dominate hidden layers, the softmax activation function serves a specialised role in output layers for multi-class classification tasks. Softmax transforms arbitrary real-valued scores into a valid probability distribution over classes, ensuring outputs sum to 1 and enabling interpretation as class probabilities. This mathematical transformation proves essential for training with cross-entropy loss and making probabilistic predictions.

Biological Inspiration

Activation functions draw loose inspiration from biological neurons, which fire (activate) only when input signals exceed certain thresholds. However, modern activation functions are primarily chosen for their mathematical properties and training characteristics rather than biological fidelity.

Computational Efficiency

The extreme simplicity of ReLU—requiring only a max operation—enables highly efficient implementations on GPUs and specialised hardware, contributing significantly to the feasibility of training massive networks on large datasets within reasonable timeframes.

Research Frontiers

Despite ReLU's dominance, research continues exploring novel activation functions. Recent work investigates learnable activations, self-gating mechanisms, and architectures that adapt activation shapes during training, seeking marginal improvements in convergence speed and final performance.

The strategic placement and selection of activation functions profoundly influences network training dynamics and final performance. Modern best practices typically apply ReLU or a variant after every convolutional layer, immediately introducing non-linearity before subsequent operations. This interleaving of linear convolutions with non-linear activations creates the repeated composition of functions that enables deep networks to learn progressively more abstract representations, ultimately transforming raw pixels into semantic understanding suitable for complex visual reasoning tasks.

Putting It All Together: Typical CNN Layer Workflow

The true power of convolutional neural networks emerges not from any single component in isolation, but from the elegant orchestration of convolution, padding, stride, pooling, and non-linearity into a unified computational pipeline. Understanding how these elements combine and interact across multiple layers reveals the architectural principles underlying state-of-the-art computer vision systems. Each layer in a CNN transforms its input through this carefully designed sequence of operations, progressively building from simple edge detectors to sophisticated object recognisers whilst managing computational complexity through strategic downsampling.



Hierarchical Feature Learning

The layer-by-layer progression through a CNN implements a form of hierarchical feature learning that mirrors, in abstract form, the visual processing hierarchy observed in biological vision systems. Early convolutional layers, operating on high-resolution inputs with small receptive fields, learn to detect simple, local features like edges, corners, and colour gradients. These low-level features combine in middle layers to form more complex patterns—textures, simple shapes, and parts of objects. Deep layers, with their large receptive fields built up through successive convolutions and pooling, learn to recognise entire objects, scenes, and abstract visual concepts, operating on highly compressed spatial representations rich with semantic information.

Balancing Resolution and Abstraction

Successful CNN architectures carefully balance spatial resolution against semantic abstraction through judicious choices of padding and stride. Early layers typically use "same" padding with stride 1 to preserve spatial detail whilst building initial feature representations. Middle layers introduce downsampling through pooling or strided convolutions, trading spatial resolution for increased depth and computational efficiency. Deep layers operate on spatially coarse but semantically rich representations, where each activation corresponds to large input regions and encodes high-level visual concepts.

Computational Efficiency

This progressive reduction in spatial dimensions proves essential for computational tractability. Without pooling or strided convolutions, feature maps would maintain input resolution throughout the network, creating prohibitive memory requirements and computational costs. Strategic downsampling enables networks to increase feature map depth—capturing more diverse patterns—whilst reducing total activation count, allowing very deep architectures that would otherwise exceed available computational resources.

Design Principles

Modern CNN architectures follow established design patterns: gradually increasing feature map depth whilst reducing spatial dimensions, applying non-linearities after every convolution, and using batch normalisation to stabilise training dynamics.

Task-Specific Adaptations

Whilst the fundamental CNN workflow remains consistent, specific architectures adapt these principles to task requirements. Segmentation networks preserve resolution longer, object detectors use multi-scale features, and generative models employ transposed convolutions for upsampling.

The Path Forward

Understanding this integrated workflow provides the foundation for both using existing CNN architectures effectively and designing novel architectures tailored to specific visual recognition challenges in domains ranging from medical imaging to autonomous vehicles.

Key Insight: The synergy between convolution, non-linearity, and pooling creates an architecture fundamentally suited to visual data, exploiting spatial structure through local connectivity whilst building abstract representations through depth. This design philosophy has proven remarkably general, extending beyond images to any spatially or temporally structured data.

The ImageNet Challenge and the Rise of Deep CNNs

The ImageNet Large Scale Visual Recognition Challenge (ILSVRC) fundamentally transformed the landscape of computer vision research by providing an unprecedented resource: a massive, meticulously labelled dataset containing over 1.2 million high-resolution images spanning 1000 distinct object categories. This ambitious project, launched in 2010, created a standardised benchmark that would push the boundaries of what was computationally possible in visual recognition tasks.

The 2012 Breakthrough

AlexNet's revolutionary victory slashed error rates by nearly 10 percentage points, achieving a top-5 error rate of 15.3% compared to the previous year's 25.8%

Proof of Concept

Demonstrated that deep learning architectures trained on massive datasets could achieve superhuman performance on complex visual tasks

Industry Impact

Sparked an explosion of research and commercial applications in computer vision, from autonomous vehicles to medical imaging

The breakthrough moment arrived in 2012 when a team from the University of Toronto, led by Alex Krizhevsky, stunned the computer vision community with their deep convolutional neural network entry. This watershed achievement didn't merely edge out competitors—it obliterated them, proving beyond doubt that deep learning held the key to practical, scalable solutions for complex visual recognition problems.

This comprehensive document explores the key CNN architectures that shaped the field following this pivotal moment. We'll examine AlexNet's groundbreaking design, ZFNet's refined approach through visualisation insights, GoogleNet's innovative multi-scale feature extraction, ResNet's elegant solution to training ultra-deep networks, and DenseNet's efficient feature reuse strategy. Each section includes practical code examples in PyTorch, enabling you to understand both the theoretical foundations and practical implementation of these influential architectures.

AlexNet: The Deep CNN That Changed Everything

Developed by Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton at the University of Toronto, AlexNet emerged as the undisputed champion of the ILSVRC 2012 competition, achieving a remarkable top-5 error rate of 15.3%—a performance that far surpassed all traditional computer vision approaches. This wasn't merely an incremental improvement; it represented a paradigm shift that would fundamentally reshape how researchers and practitioners approached visual recognition tasks.

01	02	03
Convolutional Feature Extraction	Non-linear Activation	Dense Classification
Five convolutional layers progressively learn hierarchical features from raw pixels to complex object parts	ReLU (Rectified Linear Unit) activation functions dramatically accelerated training convergence compared to traditional sigmoid functions	Three fully connected layers with dropout regularisation perform final classification across 1000 ImageNet categories

The architectural innovations within AlexNet were numerous and significant. The network comprised eight learned layers: five convolutional layers responsible for feature extraction, followed by three fully connected layers that performed the final classification. Each design choice reflected careful consideration of both theoretical principles and practical constraints. The adoption of ReLU activation functions addressed the vanishing gradient problem that had plagued deeper networks, enabling faster training and better convergence properties.

Technical Innovations	Training Strategy
- Local response normalisation for competitive inhibition between neurons - Overlapping max-pooling to reduce spatial dimensions whilst retaining information - Dropout with 0.5 probability in fully connected layers to prevent co-adaptation - Extensive data augmentation through random crops and horizontal flips	- Trained on two NVIDIA GTX 580 GPUs with split parallelisation - Stochastic gradient descent with momentum of 0.9 - Learning rate of 0.01, reduced by factor of 10 when validation error plateaued - Weight decay of 0.0005 for regularisation

Perhaps most crucially, AlexNet's success demonstrated that with sufficient data, computational resources, and architectural depth, neural networks could learn robust feature representations without manual feature engineering—a revelation that sparked the deep learning revolution across computer vision and beyond.

ZFNet: Refining AlexNet's Vision with Visualisation Insights

Building upon AlexNet's groundbreaking success, Matthew Zeiler and Rob Fergus introduced ZFNet in 2013, securing victory at ILSVRC 2013. What distinguished ZFNet wasn't radical architectural departure but rather a principled, visualisation-driven approach to understanding and optimising convolutional neural networks. By developing sophisticated techniques to visualise what individual layers learned, Zeiler and Fergus transformed CNN design from an art guided by intuition into a more scientific process informed by empirical observation.



Deconvolutional Visualisation

Developed deconvnet approach to project feature activations back to pixel space, revealing what patterns excited specific neurons



Architecture Refinement

Reduced first-layer filter size from 11×11 to 7×7 and decreased stride from 4 to 2, capturing finer low-level details



Performance Improvement

Achieved 14.8% top-5 error rate with fewer parameters than AlexNet through principled design modifications

The visualisation techniques pioneered by ZFNet revealed fascinating insights into hierarchical feature learning. Early layers detected simple patterns such as edges and colour blobs, whilst middle layers combined these into textures and object parts. Deep layers recognised complete objects and sophisticated patterns. This hierarchical representation mirrored biological vision systems, providing theoretical validation for deep learning's effectiveness.

Key Architectural Modifications

ZFNet's targeted improvements addressed specific limitations identified through visualisation:

- **Smaller first-layer filters** (7×7 vs 11×11) captured finer spatial details without aliasing
- **Reduced stride** (2 vs 4) in first layer preserved more spatial information
- **Adjusted second-layer filters** from 5×5 to 7×7 for better feature extraction
- **Maintained depth** of 8 layers whilst improving feature quality

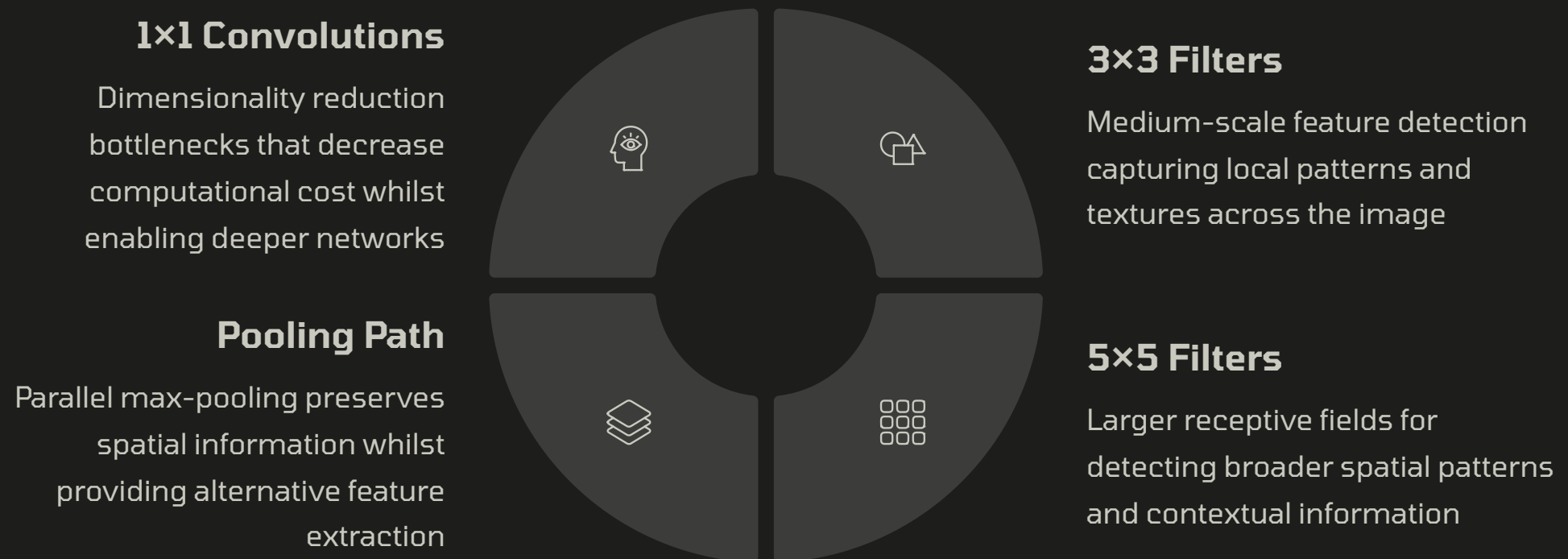
Impact on Field

ZFNet's contribution extended beyond competition victory. It established visualisation as an essential tool for understanding and debugging deep networks, inspiring interpretability research that continues today.

The success of ZFNet underscored an important principle: incremental, evidence-based improvements guided by understanding network behaviour could yield significant performance gains. This methodology influenced subsequent architecture development, encouraging researchers to adopt more systematic approaches to neural network design rather than relying solely on trial and error.

GoogleNet (Inception): The Intuition Behind Multi-Scale Feature Extraction

The 2014 ImageNet competition witnessed a paradigm shift with GoogleNet's victory, introducing the Inception architecture that fundamentally reimaged how convolutional neural networks could be structured. Developed by Christian Szegedy and colleagues at Google, this innovative design achieved state-of-the-art accuracy whilst dramatically reducing computational requirements—a seemingly contradictory achievement that stemmed from profound insights into multi-scale feature processing and network efficiency.



The core innovation lay in the Inception module—a carefully crafted building block that performed convolutions with multiple filter sizes (1x1, 3x3, and 5x5) in parallel, concatenating their outputs. This design reflected a profound intuition: objects in images appear at varying scales, and optimal feature extraction requires capturing information across multiple spatial resolutions simultaneously. Rather than forcing the network to choose a single filter size, Inception modules let the network learn which scale of features proved most discriminative for each specific task.

"The main idea of the Inception architecture is based on finding out how an optimal local sparse structure in a convolutional vision network can be approximated and covered by readily available dense components." — Szegedy et al., 2014

Computational Efficiency

Despite its 22-layer depth, GoogleNet contained approximately 5 million parameters—twelve times fewer than AlexNet's 60 million. This remarkable efficiency stemmed from three key strategies:

- 1x1 convolutions as bottleneck layers dramatically reduced dimensionality
- Global average pooling replaced fully connected layers, eliminating millions of parameters
- Auxiliary classifiers during training combated vanishing gradients

Architectural Statistics

- Total depth:** 22 layers (27 including pooling)
- Inception modules:** 9 stacked modules
- Parameters:** ~5 million
- Top-5 error:** 6.67% on ImageNet
- Inference speed:** 2× faster than AlexNet

GoogleNet's success validated several critical principles that would influence future architecture design: the importance of multi-scale feature extraction, the power of bottleneck layers for computational efficiency, and the feasibility of training very deep networks when architectural choices facilitated gradient flow. These insights paved the way for even more sophisticated designs.

ResNet: Overcoming the Vanishing Gradient with Residual Learning

The 2015 ImageNet competition marked another watershed moment when Microsoft Research's Kaiming He and colleagues introduced ResNet (Residual Networks), an architecture that shattered previous depth limitations by enabling successful training of networks with 50, 101, or even 152 layers. This achievement addressed a fundamental challenge that had vexed researchers: paradoxically, simply adding more layers to neural networks often degraded performance rather than improving it—a phenomenon that couldn't be explained by overfitting alone.

1

The Degradation Problem

Deeper networks theoretically should perform at least as well as shallower ones, yet empirically exhibited higher training error

2

Residual Learning Hypothesis

Learning residual mappings $F(x) = H(x) - x$ is easier than learning the desired underlying mapping $H(x)$ directly

3

Identity Shortcuts

Skip connections allow gradients to flow directly backwards, enabling effective training of ultra-deep architectures

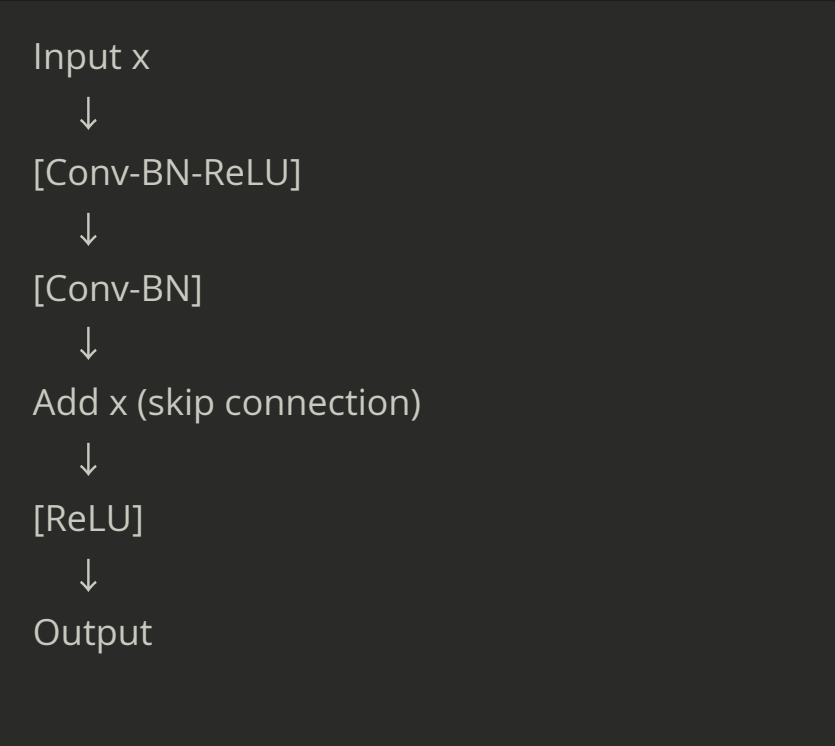
4

State-of-the-Art Results

ResNet-152 achieved 3.57% top-5 error on ImageNet, surpassing human-level performance benchmarks

The elegant solution lay in residual learning: instead of expecting each layer stack to directly learn a desired underlying mapping $H(x)$, ResNet reformulated the problem. Each residual block learned the residual function $F(x) = H(x) - x$, with the original input x added back via a skip connection. This seemingly simple modification had profound implications. If the optimal mapping was close to an identity function, the network could easily learn to set $F(x) \approx 0$ rather than struggling to approximate the identity through multiple non-linear layers.

Residual Block Structure



Each block comprises two or three convolutional layers with batch normalisation. The input bypasses these layers through an identity shortcut.

The theoretical underpinning revealed why residual connections proved so effective: they transformed the network into an implicit ensemble of shallower networks. Each skip connection created an exponential number of paths through the network, effectively creating a collection of networks of varying depths that could be trained simultaneously. This ensemble effect, combined with improved gradient flow, explained ResNet's exceptional performance.

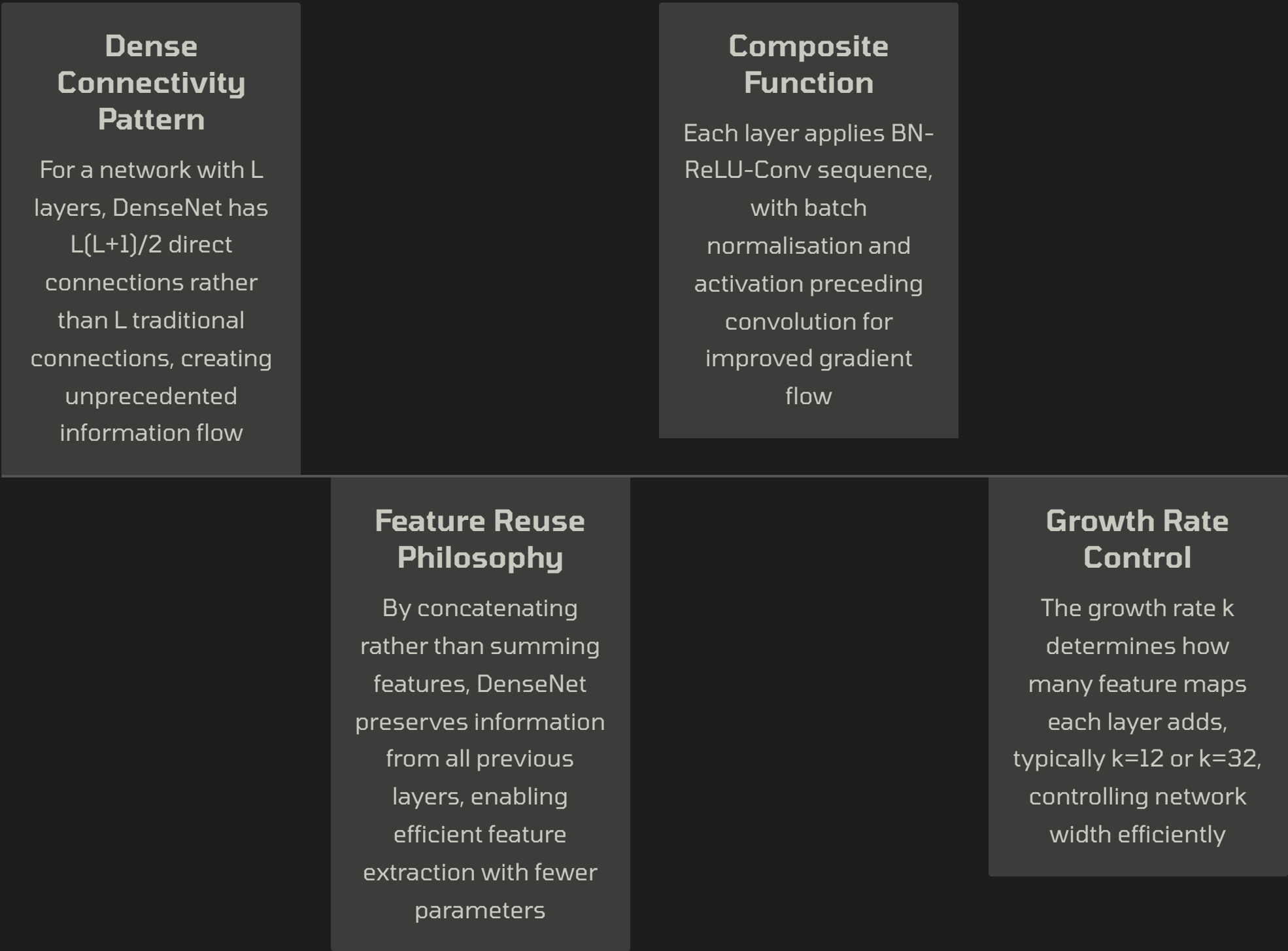
ResNet's impact extended far beyond computer vision. Residual connections became a fundamental architectural component across domains, from natural language processing to speech recognition. The principle that facilitating information flow through skip connections enables training of much deeper networks influenced virtually every subsequent architecture design, establishing residual learning as a cornerstone of modern deep learning.

Variants and Improvements

- ResNet-34:** 34 layers with basic residual blocks, 3.6% top-5 error
- ResNet-50:** 50 layers using bottleneck design with 1×1 convolutions
- ResNet-101:** 101 layers achieving 3.63% error
- ResNet-152:** 152 layers, winning architecture with 3.57% error
- ResNet-1000:** Experimental 1000-layer network demonstrating scalability

DenseNet: Maximising Feature Reuse Through Dense Connectivity

In 2017, Gao Huang, Zhuang Liu, and colleagues at Cornell University and Tsinghua University introduced DenseNet (Densely Connected Convolutional Networks), an architecture that took the concept of skip connections to its logical extreme. Whilst ResNet added previous layer outputs to subsequent layers, DenseNet concatenated them, creating a radical connectivity pattern where each layer received feature maps from all preceding layers within a dense block. This seemingly excessive connectivity yielded surprising benefits in parameter efficiency, gradient flow, and feature reuse.



The architecture organised layers into dense blocks separated by transition layers. Within each dense block, layers were densely connected following the pattern: the ℓ th layer received as input the concatenation of feature maps from all previous layers $[x_0, x_1, \dots, x_{\ell-1}]$. Transition layers between dense blocks performed convolution and pooling to reduce spatial dimensions and control feature map growth. This design created remarkably compact models—a DenseNet-BC (Bottleneck-Compression) variant with 250 layers and 15.3 million parameters achieved comparable accuracy to a 1001-layer ResNet with 10.2 million parameters.

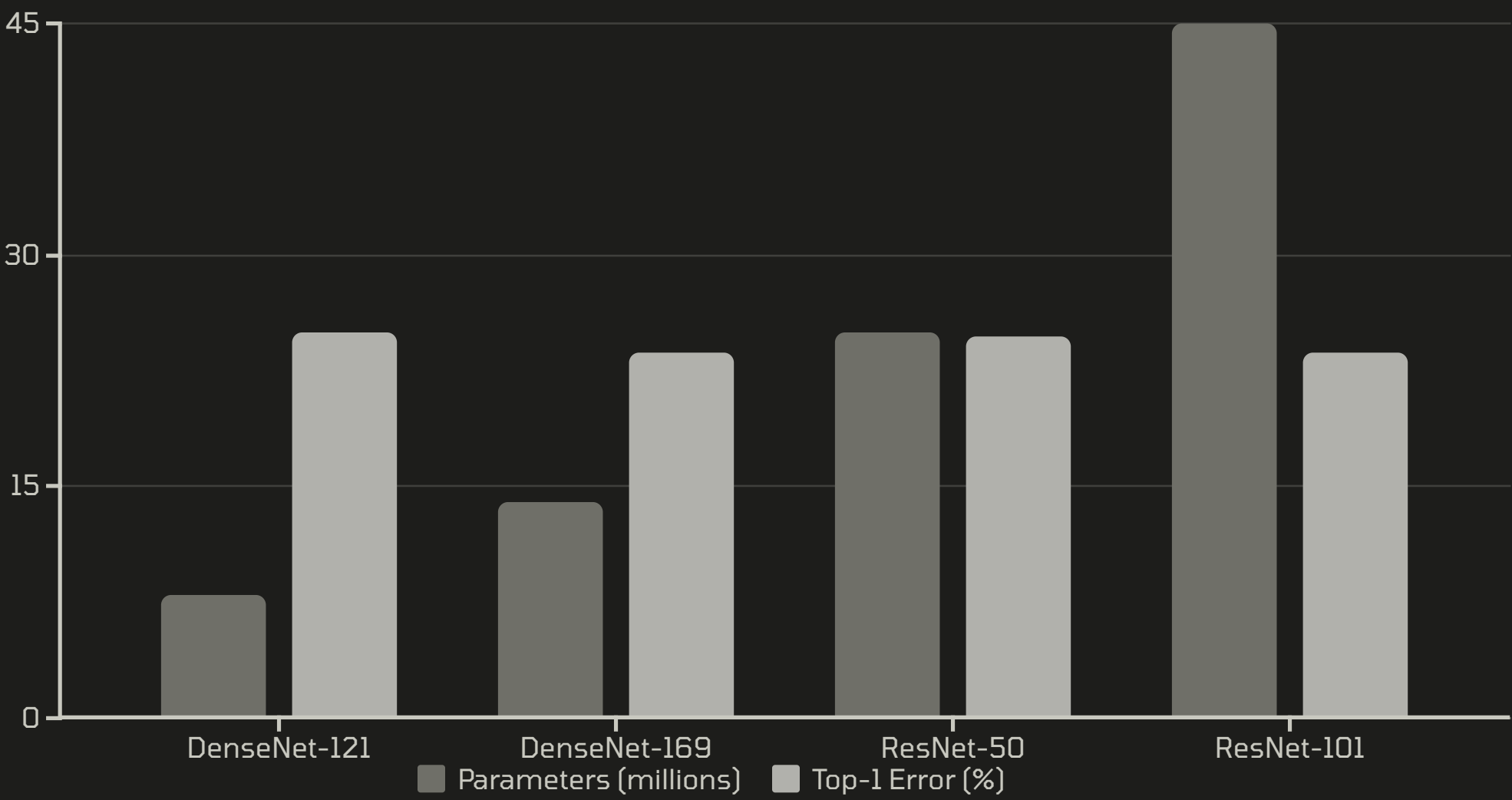
Advantages of Dense Connectivity

- Alleviated vanishing gradient:** Short paths from loss to all layers facilitated training
- Encouraged feature reuse:** Access to all previous features reduced redundancy
- Reduced parameters:** Layers learned fewer new features, relying on previous ones
- Implicit deep supervision:** All layers received supervision directly through short connections

Architecture Components

- Dense blocks:** Regions of dense connectivity with constant spatial dimensions
- Transition layers:** 1×1 conv + 2×2 pooling reducing dimensions between blocks
- Growth rate k :** Number of feature maps each layer produces (typically 12-32)
- Bottleneck layers:** Optional 1×1 convolutions reducing input features before 3×3 conv
- Compression factor θ :** Reduces feature maps at transitions (typically $\theta=0.5$)

DenseNet's parameter efficiency stemmed from an important insight: in traditional architectures, each layer learns potentially redundant features because it lacks direct access to earlier representations. By providing access to all previous features, DenseNet enabled each layer to learn only the small number of additional features needed to complement existing ones. This collective knowledge accumulation, rather than independent feature learning, explained the architecture's efficiency.



The success of DenseNet demonstrated that aggressive feature reuse through dense connectivity could achieve excellent performance with remarkable parameter efficiency—a crucial consideration for deployment in resource-constrained environments.

Practical Case Study: Implementing Classic CNN Architectures in Python

Understanding architectural principles gains practical value through implementation. This section presents PyTorch code demonstrating how to construct these influential CNN architectures, enabling you to experiment with and adapt these designs for your own computer vision projects. We'll focus on simplified but functional implementations that capture the essential architectural innovations whilst remaining accessible and educational.

AlexNet Implementation

```
import torch
import torch.nn as nn

class AlexNet(nn.Module):
    def __init__(self, num_classes=1000):
        super(AlexNet, self).__init__()

        # Feature extraction layers
        self.features = nn.Sequential(
            # Conv1: 96 filters, 11x11, stride 4
            nn.Conv2d(3, 96, kernel_size=11, stride=4, padding=2),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),

            # Conv2: 256 filters, 5x5
            nn.Conv2d(96, 256, kernel_size=5, padding=2),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),

            # Conv3: 384 filters, 3x3
            nn.Conv2d(256, 384, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),

            # Conv4: 384 filters, 3x3
            nn.Conv2d(384, 384, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),

            # Conv5: 256 filters, 3x3
            nn.Conv2d(384, 256, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),
        )

        # Classifier layers with dropout
        self.classifier = nn.Sequential(
            nn.Dropout(p=0.5),
            nn.Linear(256 * 6 * 6, 4096),
            nn.ReLU(inplace=True),
            nn.Dropout(p=0.5),
            nn.Linear(4096, 4096),
            nn.ReLU(inplace=True),
            nn.Linear(4096, num_classes),
        )

    def forward(self, x):
        x = self.features(x)
        x = torch.flatten(x, 1)
        x = self.classifier(x)
        return x

# Instantiate and test
model = AlexNet(num_classes=1000)
print(f'Total parameters: {sum(p.numel() for p in model.parameters())}.')
```

ResNet Residual Block Implementation

```
class ResidualBlock(nn.Module):
    def __init__(self, in_channels, out_channels, stride=1, downsample=None):
        super(ResidualBlock, self).__init__()

        # Main path
        self.conv1 = nn.Conv2d(in_channels, out_channels,
                                kernel_size=3, stride=stride, padding=1, bias=False)
        self.bn1 = nn.BatchNorm2d(out_channels)
        self.relu = nn.ReLU(inplace=True)

        self.conv2 = nn.Conv2d(out_channels, out_channels,
                                kernel_size=3, stride=1, padding=1, bias=False)
        self.bn2 = nn.BatchNorm2d(out_channels)

        # Skip connection with optional downsampling
        self.downsample = downsample

    def forward(self, x):
        identity = x

        # Main path
        out = self.conv1(x)
        out = self.bn1(out)
        out = self.relu(out)

        out = self.conv2(out)
        out = self.bn2(out)

        # Add skip connection
        if self.downsample is not None:
            identity = self.downsample(x)

        out += identity # Residual connection
        out = self.relu(out)

        return out

class ResNet18(nn.Module):
    def __init__(self, num_classes=1000):
        super(ResNet18, self).__init__()

        self.in_channels = 64

        # Initial convolution
        self.conv1 = nn.Conv2d(3, 64, kernel_size=7, stride=2, padding=3, bias=False)
        self.bn1 = nn.BatchNorm2d(64)
        self.relu = nn.ReLU(inplace=True)
        self.maxpool = nn.MaxPool2d(kernel_size=3, stride=2, padding=1)

        # Residual layers
        self.layer1 = self._make_layer(64, 2, stride=1)
        self.layer2 = self._make_layer(128, 2, stride=2)
        self.layer3 = self._make_layer(256, 2, stride=2)
        self.layer4 = self._make_layer(512, 2, stride=2)

        # Classification head
        self.avgpool = nn.AdaptiveAvgPool2d((1, 1))
        self.fc = nn.Linear(512, num_classes)

    def _make_layer(self, out_channels, blocks, stride):
        downsample = None
        if stride != 1 or self.in_channels != out_channels:
            downsample = nn.Sequential(
                nn.Conv2d(self.in_channels, out_channels, 1, stride, bias=False),
                nn.BatchNorm2d(out_channels)
            )

        layers = []
        layers.append(ResidualBlock(self.in_channels, out_channels, stride, downsample))
        self.in_channels = out_channels

        for _ in range(1, blocks):
            layers.append(ResidualBlock(out_channels, out_channels))

        return nn.Sequential(*layers)

    def forward(self, x):
        x = self.conv1(x)
        x = self.bn1(x)
        x = self.relu(x)
        x = self.maxpool(x)

        x = self.layer1(x)
        x = self.layer2(x)
        x = self.layer3(x)
        x = self.layer4(x)

        x = self.avgpool(x)
        x = torch.flatten(x, 1)
        x = self.fc(x)

        return x
```

Training Example

```
# Initialize model, loss, optimizer
model = ResNet18(num_classes=10)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(
    model.parameters(),
    lr=0.1,
    momentum=0.9,
    weight_decay=1e-4
)

# Training loop
model.train()
for epoch in range(num_epochs):
    for images, labels in train_loader:
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
```

Key Considerations

- Use appropriate data augmentation (random crops, horizontal flips)
- Apply learning rate scheduling (reduce on plateau or cosine annealing)
- Monitor training/validation metrics to detect overfitting
- Use mixed-precision training for faster convergence on modern GPUs
- Consider transfer learning from pre-trained ImageNet weights

These implementations provide starting points for experimentation. Modern frameworks like torchvision provide pre-trained versions of these architectures, which can be fine-tuned for specific tasks through transfer learning—often the most practical approach for real-world applications with limited training data.